

Service configuration & settings

Early Access · MessageFoundry v0.3.2 · July 2026

Status: the catalog is built. The `ServiceSettings` model + loader (`config/settings.py`) and the **CLI > env > file > default** precedence are built and wired into `serve`, along with `--service-config`. Every bracketed `[section]` catalogued below is a real, validated section on the `ServiceSettings` model — `[store]` (SQLite and the server-DB keys), `[api]`, `[inbound]`, `[delivery]` (the retry policy, queue ordering and alert thresholds an outbound inherits when it declares none), `[environments]` (`dir`; active env = `[ai].environment`), `[logging]`, `[auth]`, `[ai]`, `[retention]` (enforced by the retention/purge + SQLite-maintenance pass), and the rest — **except** `[engine]`, which has no model at all. The four sections that used to be built-but-uncatalogued now have their own entries: `[tls]` (client trust anchors, ADR 0093), `[reference]`, `[backup]` (ADR 0049) and `[dr]` (ADR 0048).

An unimplemented key is accepted *silently* — every section model is pydantic `extra="ignore"`, so a forward-looking file still loads rather than failing. The accepted-but-ignored keys, each also flagged where it is documented: the **whole** `[engine]` section (see `[engine]`), `[delivery].outbox_workers / dead_letter`, `[logging].file / max_bytes / backups`, `[retention].audit_days` (**reserved/keep-forever by design**), `[reference].max_staleness_seconds`, `[ai].baa_attested`, and `[update_check].index_url / index_allowed_hosts`.

Principle — two kinds of configuration

MessageFoundry deliberately separates them:

1. **The message graph is code-first.** Connections / Routers / Handlers are authored as Python (`config/wiring.py`) and loaded from `--config`. This never becomes a settings file — no YAML, no declarative channel config.
2. **Service/operational settings are deployment config**, not code: where the store lives and its credentials, the API bind address, logging, retention, retry defaults, etc. These are what this document covers. They're set by whoever *operates* the service (ops/admin), not by the interface author, and must keep **secrets out of source control**.

Mechanism

A single `messagefoundry.toml` (TOML — consistent with `pyproject.toml`; **not** YAML, and not channel config) with one section per group, plus **environment-variable overrides** for secrets, plus **CLI flags** for the common knobs. Precedence (highest first):

```
CLI flag > environment variable > messagefoundry.toml > built-in default
```

- File location: `./messagefoundry.toml` by default, or `--service-config <path>`.
- **Secrets** (e.g. a DB password) should come from **env** (or a secret reference), never plaintext in the file — env wins over the file so a deployment can inject them.
- Env naming: `MEFOR_<SECTION>_<KEY>` (e.g. `MEFOR_STORE_PASSWORD`, `MEFOR_API_PORT`). The parser splits the name at the **first** `_` after the prefix and matches that against a known-section list, so five built sections have **no env layer** and a `MEFOR_*` var aimed at one is dropped without a warning: `[sandbox]`, `[service]`, and the underscored `[cert_monitor]`, `[secret_rotation]`, `[update_check]`. Set those in the file.
- Loaded once at startup into a typed `ServiceSettings` (pydantic) model; the engine + store read from it. `serve` keeps its existing flags as the CLI layer.

Settings catalog

[store] — message store / DB

The keys are **implemented in `StoreSettings`**. **All three backends are built and selectable**: SQLite is the zero-dependency default; **Postgres** and **SQL Server** are production server-DB backends behind their extras. What each one supports is the **capability matrix** below — read it before assuming a feature is backend-limited.

Key	Type	Default	Notes
<code>backend</code>	enum	<code>sqlite</code>	<code>sqlite</code> · <code>postgres</code> · <code>sqlserver</code> · (later <code>mysql</code> / <code>oracle</code>) — all three implemented; see the capability matrix
<code>path</code>	str	<code>./messagefoundry.db</code>	SQLite only
<code>synchronous</code>	enum	<code>normal</code>	SQLite: <code>normal</code> / <code>full</code>
<code>group_commit_window_ms</code>	float (ms)	<code>0.0</code>	SQLite only (ADR 0055). When <code>> 0</code> a dedicated committer coroutine coalesces the grouped stage-handoff mutations (<code>enqueue_ingress</code> , <code>route_handoff</code> , <code>transform_handoff</code> , <code>mark_done</code> , <code>complete_with_response</code> , <code>dead_letter_now</code> , <code>mark_failed</code>) into one durable commit, amortizing the per-commit fsync (a large win under <code>synchronous = full</code> , muted under the default <code>normal</code>). A member waits up to this window for siblings to join before the batch commits; the claim / reference-snapshot / audit writes stay standalone (never grouped — the hash chain must not batch). <code>0</code> (the default) = off , byte-identical to the inline-commit path. Ignored by the server-DB backends, which coalesce through their connection pool instead.
<code>group_commit_max_batch</code>	int	<code>64</code>	SQLite only . Flush threshold for the group-commit committer: once this many members are enrolled in the open batch it commits immediately rather than waiting out the rest of <code>group_commit_window_ms</code> , bounding batch size + latency under load. Ignored when group-commit is off (<code>group_commit_window_ms = 0</code>).
<code>fifo_claim_batch</code>	int	<code>1</code>	all backends (ADR 0058). Max rows the INGRESS/ROUTED FIFO claim takes per commit. <code>1</code> = OFF (the workers claim one row per commit — byte-identical to before). <code>> 1</code> (clamped <code>1..64</code>) claims the contiguous due head-prefix in one commit and then processes each row in strict FIFO order with its own off-loop route/transform + separate handoff, amortizing the standalone claim commit toward 1/N. A not-due or producer-locked head still blocks the lane (strict per-lane FIFO, #285). The outbound/delivery claim is never batched. Opt-in throughput tuning (recommend <code>8 - 16</code>); size against worst-case message size, since N decrypted bodies are resident per lane between the claim and the N handoffs.
<code>fifo_claim_fold_reset</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever C). Folds the pooled claim's session <code>LOCK_TIMEOUT</code> reset into the claim batch on the clean success path at INGRESS/ROUTED (the write-less commit#2 disappears; the shielded finally-guard still runs on every non-clean exit — 1222, kept≠claimed, cancellation, any error). OUTBOUND/RESPONSE are never folded. <code>false</code> = byte-identical shipped batch + guard. Flip only after its own ADR 0114 §8 bench gate (AC-14).
<code>fifo_claim_proc</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever A). Executes the pooled claim via the two lane-family versioned procs <code>dbo.mefor_claim_fifo_heads_cid_v1</code> / <code>_dst_v1</code> (fixed-arity <code>{CALL}</code> , one JSON lanes parameter) instead of the ~3 KB ad-hoc batch. Needs database <code>COMPATIBILITY_LEVEL >= 130</code> (SQL Server 2016); fails safe to the batch, loudly , when the procs are missing, hand-edited (body-hash mismatch), or compat <code>< 130</code> — never a lane outage. A hardened split-principal deployment must <code>GRANT EXECUTE</code> on both procs to the runtime principal (the bootstrap principal owns them). <code>false</code> = byte-identical. Flip only after its own §8 gate (AC-14).

Key	Type	Default	Notes
<code>fifo_claim_prepared</code>	bool	<code>false</code>	SQL Server only (ADR 0114 sub-lever B). Stabilizes the pooled claim's statement text (one JSON lanes parameter) and retains a prepared claim cursor on store-owned dedicated connections (INGRESS/ROUTED; the non-DDL fallback lane to <code>fifo_claim_proc</code>). Logs + no-ops unless <code>fifo_claim_fold_reset</code> is on (without the fold the finally-guard's reset would evict the one-slot prepare cache every call). <code>false</code> = byte-identical. Flip only after its own \$8 gate (AC-14).
<code>encryption_key</code>	secret	—	env only (<code>MEFOR_STORE_ENCRYPTION_KEY</code>); base64 32-byte active key — when set, PHI columns (<code>raw / payload + error / last_error / detail</code>) are AES-256-GCM-encrypted at rest. Mint one with <code>messagefoundry gen-key</code> . Empty = off. See <code>PHI.md §3</code> .
<code>encryption_keys_retired</code>	secret	—	env only (<code>MEFOR_STORE_ENCRYPTION_KEYS_RETIRED</code>); comma-separated base64 decrypt-only keys kept available during a rotation until <code>messagefoundry rotate-key</code> finishes re-encrypting under the active key (ASVS 11.2.2).
<code>encryption_key_file</code>	path	—	Windows DPAPI-protected key file (WP-11d, ASVS 13.3.1) — a path produced by <code>messagefoundry protect-key</code> . When <code>encryption_key</code> is unset and this is set, the active key is <code>CryptUnprotectData</code> 'd from this file at store open, so the plaintext key never sits in the service environment. Windows-only, and the env key takes precedence when both are present. A path, not a secret — it may live in the file. Unset = use <code>encryption_key</code> (the cross-platform default).
<code>aad_bind</code>	bool	<code>true</code>	not a secret (<code>MEFOR_STORE_AAD_BIND</code>); cell binding (ASVS 11.3.3, ADR 0019). New at-rest AES-256-GCM writes use the cell-bound <code>mfenc:v2</code> writer — each value is bound to its (<code>table, column, row</code>) cell via GCM Associated Data, so a ciphertext cut-and-pasted into another cell fails the auth tag (dead-lettered <code>CipherError</code>) instead of silently decrypting. On by default (ADR 0148 GIVEN 1: the shipped configuration runs the hardened path). Setting it <code>false</code> selects the frozen <code>mfenc:v1</code> writer (byte-identical at rest) and is a loosening — <code>security_loosenings()</code> names it, so the opt-out is never silent. No effect without an <code>encryption_key</code> (the identity cipher has nothing to bind). Legacy <code>v1</code> rows still decrypt (dual-read); <code>messagefoundry rotate-key</code> upgrades them <code>v1</code> → <code>v2</code> , so the default is safe and reversible on an existing store.
<code>key_provider</code>	enum	<code>auto</code>	selects how the active/retired DEK bytes are <i>sourced</i> — never how they are used (the cipher, keyring, and <code>mfenc:v1</code> format are unchanged; ADR 0019, ASVS 13.3.3). <code>auto</code> (default) is the env-then-DPAPI ladder, byte-identical to the pre-seam behavior; <code>env / dpapi</code> pin a single built-in source; <code>aws_kms · azure_kv · gcp_kms · vault · pkcs11</code> envelope-decrypt a wrapped DEK inside an HSM/KMS/Vault (lazy optional extras — not built yet ; selecting one fails closed at <code>serve</code> , never a silent downgrade). Names a <i>provider</i> , not key material, so it is not a secret.
<code>cipher_provider</code>	enum	<code>aesgcm</code>	selects the at-rest cipher itself — distinct from <code>key_provider</code> , which only <i>sources</i> DEK bytes for the in-process cipher (ADR 0138 , ASVS 13.3.3). <code>aesgcm</code> (the default) is that in-process AES-256-GCM cipher, byte-identical to today. <code>vault_transit</code> performs the bulk encrypt/decrypt inside Vault/OpenBao Transit, so the plaintext DEK never enters engine heap; at-rest values then carry the <code>mfenc:v3:</code> marker, the local <code>encryption_key / key_provider</code> go unused (Transit holds the key), and the audit chain is keyed by Transit's <code>generate_hmac</code> — computed inside the vault, so no HMAC key enters heap either. Threaded through all three backends. Vault address / token / data-key name come from <code>MEFOR_STORE_VAULT_ADDR</code> , <code>MEFOR_STORE_VAULT_TOKEN</code> and <code>MEFOR_STORE_TRANSIT_KEY</code> (optionally <code>MEFOR_STORE_TRANSIT_AUDIT_KEY</code>). Names a <i>provider</i> , not key material, so it is not a secret. Any other value fails closed at <code>open_store</code> — never a silent downgrade to plaintext.
<code>require_encryption</code>	bool	<code>false</code>	when <code>true</code> , <code>serve</code> refuses to start without an encryption key in any environment, even a synthetic one. Off by default.
<code>allow_unencrypted_phi</code>			→ moved to <code>[security].allow_unencrypted_phi</code> (ADR 0118) — set it there; no longer accepted in <code>[store]</code> .

Key	Type	Default	Notes
<code>server , port</code>	str/int	— / 1433	server DBs (required for <code>sqlserver</code>)
<code>database</code>	str	—	server DBs (required for <code>sqlserver</code>)
<code>auth</code>	enum	<code>sql</code>	<code>sql · integrated · entra</code> (SQL Server). <code>integrated</code> connects <code>Trusted_Connection=yes</code> — the service account's Windows identity authenticates (no SQL password); the turnkey gMSA walkthrough (grant the gMSA a SQL login + run the service under it) is <code>DEPLOY-SERVER-DB.md §1.1</code> .
<code>username</code>	str	—	server DBs (required when <code>auth = sql</code>)
<code>password</code>	secret	—	env only (<code>MEFOR_STORE_PASSWORD</code>)
<code>require_managed_identity</code>	bool	<code>false</code>	delegated-identity precondition (#203, ASVS 13.2.1/13.3.2): when <code>true</code> , <code>serve</code> refuses to start (exit 2) unless the store authenticates via a managed identity — SQL Server <code>auth = integrated / entra</code> . SQLite is exempt; Postgres cannot satisfy it. Off by default. The refuse/warn split is <code>[security].enforcement</code> , not the deployment tier — <code>enforce</code> is the shipped default on <code>dev</code> and <code>staging</code> as much as on <code>prod</code> , so a staging box that turns this on and leaves <code>auth = "sql"</code> is refused , not warned; it downgrades to a warning only under <code>enforcement = warn</code>
<code>encrypt , trust_server_certificate</code>	bool	<code>true / false</code>	TLS to the DB
<code>ssl_root_cert</code>	path	—	server DBs — pin the DB server's certificate by file so a private/self-signed DB CA verifies without a machine-wide trust import, on the secure posture only (<code>encrypt = true</code> , <code>trust_server_certificate = false</code>) — it never disables verification. Postgres: an <code>asyncpg SSLContext</code> CA-bundle (chain + hostname still checked). SQL Server: the ODBC Driver 18.1+ <code>ServerCertificate</code> keyword (a leaf/exact-cert match; needs driver ≥ 18.1). Rejected for SQLite (no TLS); a missing file fails loud at load. A path, not a secret — may live in the file. See <code>DEPLOY-SERVER-DB.md §5</code> .
<code>multi_subnet_failover</code>	bool	<code>false</code>	SQL Server only — emit the ODBC <code>MultiSubnetFailover=Yes</code> keyword so a client connecting to an Always On Availability Group listener reaches the current primary promptly across subnets, instead of serially waiting out each replica subnet's DNS/TCP timeout on failover. A no-op for Postgres/SQLite (they never see the ODBC string). Off by default — only a multi-subnet AOAG needs it.
<code>pool_size</code>	int	40	server DBs — server-DB only (no-op on SQLite). The inverted-U optimum (raised from 5; do not set higher — over-provisioning is catastrophic, <code>ADR 0062</code>). Per engine: <code>engines × pool_size</code> share one <code>max_connections</code> — see <code>DEPLOY-SERVER-DB.md §3</code>
<code>connect_timeout , command_timeout</code>	int (s)	15 / 30	server DBs
<code>warm_pool</code>	bool	<code>true</code>	server DBs — pre-open pooled connections in the background on graph start/promotion so a connection burst (the post-promotion delivery workers, or a cold start) finds them warm instead of paying cold connects (TCP+TLS+login). Best-effort, self-releasing, no-op on SQLite . On by default (it touches no commit/correctness seam); set <code>false</code> to opt out on a connection-constrained/licensed site.
<code>warm_pool_timeout</code>	num (s)	15	server DBs — upper bound on the background warm-up; on expiry it logs and continues with a partially warm pool. Must be <code>> 0</code> . A clustered server-DB node also rejects an explicit value <code>>= [cluster].leader_fence_timeout_seconds</code> (a warm should finish within the leadership term that started it); the default (15 < the 20 fence) never trips this.
<code>warm_pool_target</code>	int	—	server DBs — how many connections to pre-open. Unset (default) = a safe fraction of the pool (<code>min(pool_size-1, pool_size//2)</code>), so the warm never pins more than half the pool; an explicit value is clamped to <code>pool_size-1</code> . A pool of 1 is never warmed. At the default <code>pool_size = 40</code> this is <code>min(39, 20) = 20</code> pre-opened per server-DB engine at startup.

Key	Type	Default	Notes
<code>db_schema</code> , <code>application_name</code>	str	— / <code>messagefoundry</code>	optional (<code>db_schema</code> $\Rightarrow$ env <code>MEFOR_STORE_DB_SCHEMA</code>)
<code>lease_ttl_seconds</code>	num (s)	60	server DBs — the in-flight row-lease TTL . A worker that claims a row stamps owner + <code>lease_expires_at = now + this</code> ; a renew timer extends it while processing, and the <code>[cluster]</code> leader's reclaim sweep recovers only rows whose lease has expired , so a crashed node's work comes back without stealing a live sibling's in-flight rows. The lease is wall-clock across nodes , so set it comfortably above expected clock skew + the renew interval. A shared server-DB field: SQL Server and SQLite don't lease and ignore it.
<code>uploads_dir</code>	path	—	Off unless set . Enables the opt-in uploaded-logs surface (POST <code>/uploads</code> + the <code>/ui/uploaded-logs/upload</code> delegate, ADR 0134); a filesystem dir for operator-uploaded diagnostic logs. A storage path , not a secret. Unset = no PHI-at-rest upload surface exists. See CONNECTIONS.md §"Uploaded-logs file policy" .
<code>max_upload_bytes</code>	int (bytes)	<code>26214400</code> (25 MiB)	Hard cap on a single uploaded file (<code>ge=1</code> , <code>le=512</code> MiB). Bounds the multipart upload buffer and the offline whole-file split; the global 1 MiB HTTP body cap is raised to this value only on the two upload routes.
<code>max_upload_files_per_user</code>	int	100	Max number of uploaded diagnostic files one uploader may retain at once (ASVS 5.2.4, <code>ge=1</code>). A would-be 101st upload is refused HTTP 409 (<code>upload.reject_quota</code>). Default-on once <code>uploads_dir</code> is set — the control cannot ship disabled.
<code>max_upload_total_bytes_per_user</code>	int (bytes)	<code>262144000</code> (250 MiB)	Max aggregate bytes of uploaded files one uploader may retain (ASVS 5.2.4, <code>ge=1</code>). An upload pushing the uploader's total over the cap is refused HTTP 409 . Default-on.
<code>uploads_retention_days</code>	int (days)	30	Age after which an uploaded file (blob+meta pair) is pruned (ASVS 5.2.4, <code>ge=1</code>) — swept opportunistically at save time and by a periodic task; every prune audited (<code>upload.prune</code> , <code>id</code> + uploader only). Default-on.

Selecting `backend = "sqlserver"` validates that `server` / `database` (and `username` when `auth = "sql"`) are present. The backend is **production** (full staged pipeline, response capture, at-rest encryption): it needs the `sqlserver` extra (`pip install 'messagefoundry[sqlserver]'`) plus the Microsoft ODBC Driver 18, and is exercised against a real SQL Server by the CI service-container job. SQLite remains the zero-dependency default.

Per-backend capability matrix

Each row is a `supports_*` capability flag on the `QueueStore` protocol (`store/base.py`); each cell is the value the backend's store class actually declares. The engine reads these flags to **fail closed at startup** — an unsupported feature is refused before any message is accepted, never a silent degrade and never a post-ACK surprise.

Capability flag	SQLite	Postgres	SQL Server
<code>supports_ingest_stage</code>	yes	yes	yes
<code>supports_response_capture</code>	yes	yes	yes
<code>supports_pt_reingress</code>	yes	yes	yes
<code>supports_streaming_attachments</code>	yes	yes	yes
<code>supports_fused_sync_handoff</code>	no	no	yes
<code>supports_reference_sets</code>	yes	yes	yes

Request/response capture (ADR 0013), PT/ Loopback() re-ingress, and ADR 0006 reference sets work on ALL THREE backends — including SQL Server, which has shipped `capture_response` + `reingress_to` at full parity since #249 and the reference-snapshot store since [BACKLOG #235](#) (2026-07-16, CI-proven against real SQL Server 2022 + 2025). Do not read a backend limitation into any of those rows. (The reference-set gate itself stays: a graph declaring a `Reference(...)` on a *future* backend that leaves the allow-list default `False` is still refused at `messagefoundry check`, at engine start, and on reload/promote.)

The one row that *does* vary:

- **supports_fused_sync_handoff** — **SQL Server only**. The fused synchronous handoff twins (ADR 0071 B5) collapse a multi-statement handoff into one executor completion. The profiled wall is aiiodbc's per-statement thread crossing, which only SQL Server pays: asyncpg is loop-native and SQLite's handoff lock is loop-affine, so neither has anything to fuse. SQL Server is the *most* capable backend here.

This table is pinned by `tests/test_store_capability_matrix.py`, which parses it and asserts every cell against the live store-class attributes. Flip a flag or add a new one and you must update this table **in the same commit**, or the test fails. That is deliberate: the stale "backend X doesn't support Y" prose this table replaced once sent a team off to build a feature that already existed.

[api]

Key	Type	Default	Notes
host			→ moved to <code>[security].local_access_only / listen_address</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code> .
port	int	8765	
expose_docs	bool	false	serve <code>/docs</code> , <code>/redoc</code> , <code>/openapi.json</code> (off by default — widens surface)
config_reload_roots	list[str]	[]	extra directories <code>POST /config/reload</code> may load from, besides the startup <code>--config</code> dir. The loader executes Python from these, so list only admin-owned, trusted roots (e.g. an IDE staging dir). Any reload path outside the startup dir + these roots is rejected (403).
tls_cert_file	str	unset	[BUILT] (WP-13a, ADR 0002) : PEM server-certificate path. Setting it turns on in-process TLS — the API serves <code>https / wss</code> , HSTS engages, and a non-loopback bind is allowed without <code>--allow-insecure-bind</code> .
tls_key_file	str	unset	PEM private-key path (omit if the key is in the cert PEM). Requires <code>tls_cert_file</code> .
tls_key_password	secret	unset	passphrase for an encrypted key — env only (<code>MEFOR_API_TLS_KEY_PASSWORD</code>), never the file.
tls_min_version	str	1.2	minimum negotiated TLS version floor (NIST SP 800-52r2): <code>1.2</code> or <code>1.3</code> .
tls_ciphers	str	unset	optional OpenSSL cipher string (default = the interpreter's secure defaults).
tls_client_ca_file	str	unset	CA bundle to require + verify client certs (opt-in mTLS, e.g. the console). Requires <code>tls_cert_file</code> .
tls_client_ca_pin	str	unset	optional lowercase-hex SHA-256 pin over the corresponding CA anchor PEM (<code>tls_client_ca_file</code>); a mismatch refuses at load + reload (ASVS 6.7.1); unset = no pin (dormant).
tls_client_cert_identities	map str→str	{}	(#200, ADR 0002) : mTLS client-cert → MessageFoundry principal map. Meaningful only with in-process mTLS (<code>tls_client_ca_file</code> set, so uvicorn <code>CERT_REQUIRED</code> -verifies the peer): a verified peer cert's subject CN / SAN is resolved to an existing username through this allow-list , and that principal's RBAC authorizes the request — a service-to-service identity that carries no bearer token. Keys are the qualified cert name <code>CN:<commonName></code> or <code>SAN:<type>:<value></code> (e.g. <code>SAN:DNS:svc.internal</code>); values are existing usernames. Deny-by-default : an unmapped verified cert — or a spoofed CN not listed here — resolves to no identity and is denied. A structured map, so TOML-only (no env-string form). Empty (the default) disables cert-identity. Honest caveat : stock uvicorn does not surface the peer cert to the ASGI scope, so the resolver is inert until a TLS-extension-capable server/shim populates it.
tls_client_cert_files	list[str]	[]	(ASVS 6.4.5) : PEM paths of inbound service callers' client certs you hold a copy of. Folded into the <code>[cert_monitor]</code> scan, so a caller's cert expiry is caught even while that caller has stopped connecting — the handshake-time check can only see a cert still being presented. These are certs the engine <i>verifies</i> , not ones it <i>presents</i> , so the served-cert scan cannot see them. Public certificates only (never a key); empty = off.
trusted_proxies	list[str]	[]	[BUILT] (WP-15) : reverse-proxy IP(s) whose <code>X-Forwarded-For / -Proto</code> are trusted (uvicorn <code>forwarded_allow_ips</code>), so the audit/rate-limit source IP is the real client , not the proxy. Empty = trust nothing (the direct TCP peer is used). Set ONLY to the proxy's address(es), or XFF spoofing returns — every host inside an entry may declare its own source address, so a broad range (e.g. <code>10.0.0.0/8</code> on a LAN numbered out of 10/8) makes every workstation a trusted spoofer. <code>***</code> and unparseable entries are refused at load (uvicorn would silently treat the latter as a never-matching literal, collapsing every client to the proxy).

Key	Type	Default	Notes
<code>tls_terminated_upstream</code>	bool	<code>false</code>	[BUILT] (WP-15): declare that a reverse proxy / load balancer terminates TLS in front of the engine. Lets a non-loopback bind satisfy the TLS gate without in-process TLS — but only when <code>trusted_proxies</code> is set (else refused at load).
<code>proxy_intra_service_auth</code>	enum	<code>none</code>	Posture-B operator attestation (#200, ADR 0002) — <i>how</i> the proxy→engine hop is authenticated, so a rogue peer on the internal segment cannot impersonate the proxy. <code>none</code> (the default) is undeclared ; declare <code>mtls</code> (the proxy presents a client cert), <code>network</code> (an isolated proxy↔engine segment / host firewall allow-list) or <code>shared_secret</code> (a pre-shared header the proxy injects). Attestation only — the engine enforces nothing at run time ; it is the record that the hop was considered. Left undeclared under <code>tls_terminated_upstream</code> on a PHI instance, <code>serve</code> refuses when <code>[security].enforcement = enforce</code> and the bind is non-loopback, and warns otherwise (including the recommended loopback-behind-proxy topology).
<code>proxy_tls_min_version</code>	str	<code>unset</code>	the operator- declared TLS version floor the reverse proxy negotiates with browsers: <code>1.2</code> or <code>1.3</code> (NIST SP 800-52r2) — any other value is refused at load. The engine terminates no browser TLS in Posture-B, so it cannot inspect the proxy's negotiated version (ASVS 11.6.2); this is the attested floor, validated only for coherence. Unset = undeclared, gated exactly like <code>proxy_intra_service_auth</code> above.
<code>proxy_tls_ciphers</code>	str	<code>unset</code>	an optional declared OpenSSL cipher list for that proxy floor. When set it must resolve to forward-secret (EC)DHE suites (ASVS 11.6.2) — the same validator <code>tls_ciphers</code> uses — so a declared floor can't itself name a non-forward-secret key exchange. Unset = no cipher declaration; it is not required to satisfy the Posture-B gate (only <code>proxy_intra_service_auth</code> + <code>proxy_tls_min_version</code> are).
<code>serve_ui</code>			→ moved to <code>[security].serve_web_console</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code> .
<code>serve_ui_explicit</code>	bool	<code>false</code>	Internal plumbing — not an operator knob. The loader sets it <code>true</code> when <code>[security].serve_web_console</code> was explicitly provided (at either value), so <code>serve</code> can tell an explicit <code>serve_web_console = true</code> with the console wheel absent (hard refuse — the operator asked for the console by name) from the default-on posture with the wheel absent (JSON-only <code>serve</code> + a warning, never a start failure). Set <code>[security].serve_web_console</code> , not this.
<code>public_origin</code>			→ moved to <code>[security].web_console_public_address</code> (ADR 0118) — set it there; no longer accepted in <code>[api]</code> .
<code>ws_allowed_origins</code>	list[str]	<code>[]</code>	browser <code>Origin</code> allowlist for the native/bearer <code>/ws/stats</code> path only — NOT the <code>/ui</code> browser WebSocket (that authorizes via the cookie + <code>public_origin</code> /Host match). Don't conflate the two knobs.

MLLP-over-TLS is built too (WP-13b — per-connection `tls/tls_*` on the `MLLP(...)` connector, see [CONNECTIONS.md](#)), and the `$0` **exposed-gate is enforced**: a non-loopback *plaintext* MLLP listener is refused at startup — `serve --allow-insecure-bind` is **clamped inert** on the shipped posture and does not lift it. Gate #4's transport-TLS subset is complete, and **native TOTP MFA (WP-14) is also built** (`[security].require_mfa`, local accounts — the old `[auth].require_mfa` spelling is **rejected at config load**). See [ADR 0002](#).

WebAuthn passkeys (WP-14b, ADR 0068). Browser passkeys for local users need the optional `[webauthn] extra` (`pip install messagefoundry[webauthn]`) — no new `[auth]` setting: installing the extra + a user enrolling on `/ui/account` is the opt-in (extra-less installs show a legible notice, never an error). The WebAuthn RP identity rides the external origin — set it as `[security].web_console_public_address` (the internal field is still `api.public_origin`, but `[api].public_origin` is a relocated key and is **rejected at config load**, row above). A plain loopback deployment derives the RP from the request URL, but **behind a declared reverse proxy** (`tls_terminated_upstream`) an unset origin is a startup **refusal**, not a degraded ceremony: with the console served, `serve` exits 2 until it is set — and note that **changing that host later invalidates every enrolled passkey** (they pin their mint-time RP; the account page marks them "unusable (origin changed)").

Off-loopback browser-console walkthrough (L5b, ADR 0068 §8). The two supported postures: **in-process TLS** (`tls_cert_file` [+ `tls_key_file`]) — the browser connects directly to the engine — or a **declared upstream terminator** (`tls_terminated_upstream = true` + `trusted_proxies = ["<proxy egress IP or CIDR>"]` + `[security].web_console_public_address`, which the L5b ladder now requires). `trusted_proxies` entries match the proxy's **direct TCP peer address exactly** (CIDR supported, but scope it to the proxy pool — every host inside an entry may forge its own source address; watch the `::1`-vs- `127.0.0.1` mismatch) — a *syntactically valid but wrong* entry silently disables the forwarded-header rewrite, collapsing audit/rate-limit source IPs to the proxy. An **unparseable** entry, or `"*"`, is refused at config load rather than degrading silently. With either posture declared (`exposure_protected`), the `/ui` session cookie ships `Secure` and HSTS is emitted regardless of the per-request scheme. Full runbook + reverse-proxy-mTLS reference configs: [security/OFF-LOOPBACK-DEPLOYMENT.md](#) — a maintainer-internal document; see [SECURITY-DOCS-POLICY.md](#) for what is withheld and what you can request.

[tls] — outbound client trust anchors

Instance-wide **client trust-anchor policy** (#190, [ADR 0093](#)) — a small shared fallback for the outbound connectors that verify a downstream *server* certificate (MLLP, DICOM, FTPS today). By default the OS trust store roots verify the peer; a hospital estate whose internal endpoints present a private/internal-CA certificate can pin that CA **once** here instead of installing it box-globally or repeating a per-connection `tls_ca_file`. This selects **which** roots verify the peer — it **never disables verification** — so it composes with (never weakens) the connectors' fail-closed no-CA / `tls_verify=false` / cleartext-hop refusals. A connection naming its **own** `tls_ca_file` always wins verbatim, and a loopback hop is exempt. The default is a no-op: a config with no `[tls]` block builds a byte-identical SSL context.

Key	Type	Default	Notes
<code>internal_ca_file</code>	path	—	PEM path to the org's internal CA. Not a secret — a path, like <code>tls_cert_file</code> / <code>forward_tls_ca_file</code> . Unset (default) = no internal anchor; every hop uses the OS trust store.
<code>trust_anchor_mode</code>	enum	<code>system</code>	how <code>internal_ca_file</code> composes with the OS default roots on a non-loopback internal hop. <code>system</code> (default) = OS trust store only , <code>internal_ca_file</code> ignored (byte-identical). <code>augment</code> = OS roots and the internal CA (a mixed public + private estate). <code>pinned</code> = only the internal CA, not the public bundle (a fully-private estate; strictest). <code>pinned without internal_ca_file</code> is refused at load — with nothing to pin it would silently fall back to the full OS trust store, i.e. the operator excludes public roots and gets all of them.

[inbound] — inbound listener defaults

Key	Type	Default	Notes
<code>bind_host</code>	str	<code>127.0.0.1</code>	the default network interface every inbound MLLP/TCP listener binds to. Authors never set a <code>host</code> on an inbound connection (a wiring error if they do) — it's a per-environment operator decision here. Binding <code>0.0.0.0</code> exposes unauthenticated MLLP to the network, so it's deliberate (DEV typically loopback, PROD a specific NIC behind a firewall). A non-loopback bind requires <code>tls=true</code> on each MLLP connection: the \$0 exposed-gate refuses a plaintext off-loopback listener at startup with a <code>WiringError</code> . <code>serve --allow-insecure-bind</code> downgrades that refusal only when the instance is not both enforcing and PHI — and since <code>enforcement = enforce</code> is the default and all three built-in env names (<code>dev</code> / <code>staging</code> / <code>prod</code>) derive PHI, on a stock instance the flag is clamped inert and the bind still fails. A single connection may override this with a per-connection <code>bind_address</code> (and restrict peers with <code>source_ip_allowlist</code>) — MLLP/TCP only; see <code>CONNECTIONS.md</code> .
<code>ack_after</code>	enum	<code>ingest</code>	the default ACK timing every inbound inherits (staged pipeline, <code>ADR 0001</code>). <code>ingest</code> = ACK-on-receipt, once the raw message is durably committed to the ingress stage and before routing/transform/delivery. <code>delivered</code> (defer the ACK until delivery succeeds) is not built — wiring it raises a <code>WiringError</code> , so it fails loud rather than silently ACKing early. A connection's own <code>ack_after=</code> overrides this.
<code>stream_inflight_budget_bytes</code>	int (bytes)	<code>0</code>	aggregate cap on the total bytes of over-threshold message bodies concurrently mid-detach across all inbounds (#149, <code>ADR 0105</code>). A detach that would push the running total over it is refused with backpressure (the message is NAK'd/ <code>ERROR</code> 'd, never accepted-and-dropped), so a burst of very large documents can't exhaust memory. <code>0</code> (default) = unlimited — a <i>single</i> body is still bounded by the per-connection <code>max_message_bytes</code> . Only over-threshold streaming detaches count against it.

[environments] — per-environment graph values (DEV/PROD)

The **same** code-first graph runs in every environment; only the values it references via `env("key")` differ. The **active** environment is the single cross-cutting selector `[ai].environment` — a **free-form name** (`ADR 0017`), set in the TOML or via `serve --env <name>`, and **required** (no default); this section only locates the value files.

Key	Type	Default	Notes
<code>dir</code>	str	<code>environments</code>	directory holding <code><env>.toml</code> flat key→value tables for non-secret values, versioned in the repo. Resolved against <code>base_dir</code> (below).
<code>base_dir</code>	str	<code>""</code> (= the working dir)	Anchor <code>dir</code> resolves against. Empty keeps the original behavior (relative to the process working directory). Set it to the config-repo root so env-value resolution no longer depends on where <code>serve</code> was launched. A relative value is taken against the working dir; an absolute value is used as-is — on Windows it must be drive-qualified (<code>C:/repo</code>); a leading-slash <code>/repo</code> is drive-relative and still inherits the launch drive (logged as a warning). Overridable per run with <code>serve --project-root</code> .

- A graph value that differs by environment is authored as `env("acme_adt_host")`; the running instance resolves it from `<base_dir>/<dir>/<active-env>.toml` overlaid by `MEFOR_VALUE_<KEY>` env vars (secrets — never the file; env wins). Keys are `lower_snake_case`.
- Anchoring the value files** (`base_dir` / `--project-root`). A standalone **config repo** (`ADR 0017`) keeps `environments/` at its root — a *sibling* of the `--config` dir. With the default (empty) `base_dir`, the files resolve relative to the **process working directory**, so a `serve` launched from anywhere but the repo root reads **no** env values (a silent empty table, not an error — the missing values then fail loud only when a connector is built). This bites most under **NSSM**, whose working directory is rarely the repo. Pin the anchor so resolution is launch-independent — in the instance's `messagefoundry.toml`: `toml [environments] base_dir = "C:/srv/acme-config" # the config-repo root; environments/<env>.toml live under it` or per run: `messagefoundry serve --config config --env prod --project-root C:/srv/acme-config` (the flag overrides `[environments].base_dir`; precedence is CLI > env > file > default, like every service setting). The startup log prints the **resolved** `environments/<env>.toml` path so you can confirm where values are read from. Running from the repo root keeps working unchanged (the empty default is the working dir).
- A referenced key that is **undefined for the target environment** makes the engine refuse to load or promote that graph (fail loud) — never a silent blank host. See the env files under `environments/` and `samples/config/IB_ACME_ADT.py` for a worked example.

- **Per-face logic inside a transform:** `env()` is a *deferred reference* resolved only when a **connection** spec is built — using it in a handler is an always-truthy object (a bug). To branch a Router/Handler on the deployment, read the active environment **name** with `current_environment()` (the free-form name, e.g. "prod" / "test", or `None` in a dry-run): `python from messagefoundry import current_environment # Corepoint: If ActiveFace="Test" Then MSH-11.1 = "T" if current_environment() in ("staging", "dev"):` `msg.set("MSH-11.1", "T")` The active environment is a deployment constant, so the read is pure + re-run-safe.

Code sets — reference lookup tables (`codesets/`)

A code-first Router/Handler often needs a **reference table** — an Epic diet code → a food-service system value, a facility code → a downstream mnemonic. Rather than a hand-maintained Python dict, drop the table in a **code set** and look it up with `code_set("name")`.

- **Where.** Files live in `codesets/` **relative to the `--config` dir** — a config bundle carries its own reference tables and they **reload with the graph** (`POST /config/reload`). This is distinct from `environments/` (cwd-level endpoint values for `env()`). A missing `codesets/` dir is fine (no code sets). The code-set **name** is the file's stem (`codesets/epic_diets.csv` → "epic_diets").
- **CSV** (`<name>.csv`) — a header row; the **first column is the lookup key**. One other column → the value is that scalar (`str`); several other columns → the value is a `dict {header: cell}`. A duplicate key is a **load error** (fail loud).
- **TOML** (`<name>.toml`) — a flat table `key = value` → `{key: scalar}`; a nested `[key]` table → `{key: {...}}` (mirrors the `environments/<env>.toml` shape).
- **Usage.** Capture once at a module's top level (preferred) or look it up at call time inside a handler — both resolve: `python from messagefoundry import code_set, handler, Send`

```
DIET = code_set("epic_diets") # frozen, read-only mapping; captured at import
```

```
@handler("to_dietary") def handle(msg): msg["ODS-3"] = DIET.get(msg["ODS-3"], "") # .get(key, default) — blank on a miss fac =
code_set("facility_mnemonics").get(msg["MSH-4"]) # call-time lookup also works ... return Send("OB_DIETARY", msg) `` A CodeSet is a
read-only Mapping : cs[key] (raises KeyError naming the set on a miss), cs.get(key, default), key in cs, len(cs), iteration. It is
**frozen** — one instance is shared across transforms, so a handler must never mutate the reference data. - **Fail
loud.** code_set("missing") (no such file) or a malformed/duplicate-key CSV/TOML raises a WiringError, surfaced
by validate / messagefoundry check / reload exactly like a missing env() value — never a silent empty table. - **Purity caveat.**
The lookup is pure (key in → value out), so it's compatible with the staged pipeline's **pure-re-run** invariant ([ADR 0001]
(adr/0001-staged-pipeline-architecture.md) / CLAUDE.md §2). The one caveat: a hot-reload that **changes** a table between a run
and a crash-re-run can make the re-run derive a different output. That's acceptable for reference data (a code set is
deliberately operator-editable, and a reload is an explicit, audited act), but it is the one way a transform's re-run can
legitimately differ — note it where you document the transform. - **Editing — by hand or from the IDE.** A code set is a
plain codesets/.csv you can edit in any editor, **and** a GUI-manageable artifact ([ADR 0033](adr/0033-gui-manageable-code-
sets.md)). The VS Code extension opens a **grid editor** (rows × columns of strings — the first column is the lookup key) to
**create / edit / rename / delete** a translation table; it shells a new **messagefoundry codeset** CLI that owns validation
and the atomic write. Both editors write the same file (CSV-first), so a hand edit and a GUI save are interchangeable —
mirroring the connections editor ([ADR 0007](adr/0007-gui-manageable-connections-toml.md)). - messagefoundry codeset list --config
DIR — summarize every set under codesets/ ( .csv **and** .toml ; TOML sets are summarized and shown **read-only** in the grid —
TOML-in-grid editing is a fast-follow). - messagefoundry codeset show --config DIR --name N — the grid (headers + rows). -
messagefoundry codeset upsert --config DIR --data '{...}' — validate → write codesets/N.csv atomically (temp + replace, owner-only
perms) → **re-load the written file as the final check** ; a bad save rolls back, so the CLI never leaves an unloadable table. -
messagefoundry codeset rename --config DIR --name N --to M / ... remove --config DIR --name N`.
```

The CLI is **offline** (no engine start, no egress check — a code set is standalone data); it validates against the **same loader** that runs at startup, and the operator-supplied **name is treated as untrusted data** (rejecting path separators, `..`, absolute/drive paths, and an embedded extension, so a name can't escape `codesets/`). Apply a change with the existing audited promote/reload below. - **Promote to apply (rename/remove caveat).** Editing a `codesets/` file changes nothing live; the running graph adopts the change only through `POST /config/reload` (the IDE promote), exactly like a connection or handler change. **Renaming or removing a code set can break a handler reference** — a `code_set("old_name")` call then raises at run time (that message's `ERROR` disposition). A plain `validate` only confirms each file parses, so it **won't** catch a now-dangling reference; **run messagefoundry check after a rename/remove**, whose dry-run executes the transforms and surfaces the broken `code_set(...)` lookup before you promote. See `docs/CODESETS.md` for the full grid editor + CLI reference.

Transform state — cross-message correlation (ADR 0005)

Where code sets are **read-only** reference data, **transform state** is **read/write** correlation data a Handler accumulates across messages: an anonymous-patient mapping (persist a real MRN → a stable anonymized id and reuse it on later messages), order→result correlation, running aggregates. It is authored against two surfaces from `messagefoundry`:

```
from messagefoundry import handler, Send, SetState, state_get

@handler("anonymize")
def anonymize(msg):
    mrn = msg["PID-3.1"]
    anon = state_get("patient_anon", mrn)          # synchronous read; None on a miss
    ops = []
    if anon is None:
        anon = derive_anon_id(mrn)                # deterministic derivation preferred (see below)
        ops.append(SetState("patient_anon", mrn, anon))
    msg["PID-3.1"] = anon
    return [Send("OB_DOWNSTREAM", msg), *ops]      # Sends and SetStates, mixed in one list
```

- **Write contract — declared, never imperative.** A Handler returns `Send | SetState | list[Send | SetState] | None`; it does **not** mutate state directly. Each `SetState(namespace, key, value)` (the `value` must be JSON-serializable — validated at construction) is an **upsert by (namespace, key)** the engine applies **inside the routed→outbound handoff transaction**. `Send`-only Handlers are unchanged — fully **backward compatible**.
- **Exactly-once / re-run safety.** Because the write commits in the **same transaction** as the outbound rows, a crash before commit leaves **no** state (atomic with the handoff) and the attempt that commits applies the write **exactly once per message** — this preserves the staged pipeline's **pure-re-run** invariant (ADR 0001 / CLAUDE.md §2). A non-deterministic value (a random anon id) is still safe because only the committed attempt persists, but **prefer a deterministic derivation** where cross-run identity matters.
- **Read — synchronous, read-through cache.** Handlers are pure synchronous functions and a DB read is async, so `state_get(namespace, key, default=None)` reads an in-memory **read-through cache** the engine maintains (loaded at startup, updated as writes commit) and publishes around each router/transform run — exactly how `code_set()` resolves against an active set. A missing key returns `default` (state is sparse, not a referenced table). **Non-linearization caveat:** a read reflects committed state as of its invocation, but is **not** linearized with a concurrent sibling handler's write — fine for read-mostly correlation; a race-sensitive read-modify-write within one namespace needs author care.
- **Encryption at rest.** State values may carry PHI (MRN→id), so they are AES-256-GCM-encrypted with the store cipher just like `messages.raw`, and covered by key rotation (`messagefoundry rotate-key`).
- **Retention (TTL).** Set `[retention].state_max_age_days` to age out stale entries (a global age purge; per-namespace policy is a follow-up). Off by default = keep forever. The whole-table cache assumes **bounded** state — unbounded estates (every MRN ever seen) are a documented follow-up (ADR 0005).
- **SQL Server.** State writes ride the staged `transform_handoff`, which is implemented on the SQL Server backend, so the `state` table is **live** (parity with SQLite/Postgres); the read-through cache refreshes post-commit. Cross-node state convergence is N/A (single-node backend).

`state_get` also resolves in **dry-run** / the IDE Test Bench / `messagefoundry check`: each simulated message gets a fresh in-memory view that accumulates that run's own declared writes (so a later handler sees an earlier one's `SetState`), and `dryrun` output lists the declared state ops — **PHI-gated** behind `--show-phi` like a message body.

Reference sets — external-data enrichment (ADR 0006)

Where a **code set** is a static lookup table shipped in the bundle and **transform state** is read/write correlation, a **reference set** is **external data materialized off the message path**: a provider directory, a DB-backed translation table (the Corepoint Data Point / DB Association pattern). The engine syncs the source into a **versioned, encrypted store snapshot** on a cadence; a Handler reads it **purely** at run time. Because the read carries no external call, the staged pipeline's pure-re-run invariant holds (the only non-determinism is a snapshot flip landing between a run and a crash-re-run — the same accepted caveat as a code-set hot-reload).

- **Declare** a set in a wiring module (registers it into the graph, like `inbound`): `python from messagefoundry import Reference, FileRef, env, handler, Send, reference`

```
Reference("provider_npi", source=FileRef(path=env("provider_npi_csv")), refresh_seconds=3600)
```

```
@handler("enrich") def enrich(msg): npi = reference("provider_npi").get(msg["PV1-7.1"]) # pure dict lookup, no I/O if npi: msg.set("PV1-7.13",
npi) return Send("OB_DOWNSTREAM", msg) `` - **reference(name)** returns a frozen, read-only ReferenceSet ( rs[k] / rs.get(k, d) / k
in rs ). A missing **key** returns the default (external data is sparse); a missing/unsynched **set** raises (fail loud) at run
time → that message's ERROR disposition. Call it **inside a Handler/Router**, not at module top level (the snapshot exists only
once the store is open + synced – unlike code_set ). - **Sources:** FileRef(path=..., encoding=...) – a local CSV/TOML in the **code-
set format**, re-read on the refresh cadence (the path for an externally-produced export; path may
be env() ). DatabaseRef(server=..., database=..., statement=..., key_column=..., value_column=...) – the engine runs a read-only SQL query
on the cadence (SQL Server via the [sqlserver] extra, **production / supported**); secrets via env() ; the dial-out is gated by the
fail-closed [egress].allowed_db allowlist). key_column is the lookup key; value_column (if set) is the value, else the value is a
dict of the other columns. - **Sync.** The engine's ReferenceSyncRunner materializes each set once at startup (before listeners
serve, so reference(...) resolves on the first message) and every refresh_seconds . A source failure is **isolated**: it's logged +
alerted and the **last-good snapshot is kept** (the write isn't attempted), so one bad source never blocks the others or the
message path. - **At rest:** snapshot values are AES-GCM-encrypted (they may carry PHI) and covered by key rotation, exactly
like state /message bodies; the fail-closed [egress].allowed_db allowlist gates the DatabaseRef source's dial-out. The snapshot
store ships on **all three backends** – SQLite, Postgres, and SQL Server ([BACKLOG #235](BACKLOG.md), 2026-07-16). - **
[reference] settings:** the sync cadence + startup behaviour – catalogued in [ [reference] ](#reference) immediately below. - **Dry-
run / check** resolve file-backed sets best-effort (literal paths) so a reference-using transform validates; DB-backed or env()-
path sets are absent in a pure dry-run.
```

[reference]

The sync knobs behind the reference sets above (ADR 0006 Tier 1), enforced by the engine's `ReferenceSyncRunner` (pipeline/reference_sync.py). The runner is a **no-op when no sets are declared**, so these defaults are safe on an existing deployment.

Key	Type	Default	Notes
refresh_interval_seconds	float (s)	3600	base cadence the sync loop ticks at; each set re-materializes when its own <code>refresh_seconds</code> is due. Must be <code>> 0</code> (refused at load otherwise).
sync_on_startup	bool	true	materialize every declared set once at startup, before inbound listeners begin serving, so a transform's <code>reference(...)</code> resolves on the very first message. Strongly recommended on.
max_staleness_seconds	float (s)	0	reserved / not enforced — the intended freshness guard (alert/refuse when the active snapshot is older than this). Accepted so a forward-looking file still loads; <code>0</code> = off. Must be <code>>= 0</code> .

[auth] — authentication & RBAC

Implemented (see SECURITY.md). Authentication is **required** by default; the AD bind password is a **secret** supplied via env (MEFOR_AUTH_AD_BIND_PASSWORD), never the file. The full inventory of resource-demanding functionality the `*_rate_limit_*` throttles below defend — including the surfaces that remain **unbounded** at this release — is security/THREAT-MODEL.md §Resource-demanding functionality (ASVS 15.1.3), a maintainer-internal document (SECURITY-DOCS-POLICY.md).

Key	Type	Default	Notes
enabled			→ moved to [security].require_sign_in (ADR 0118) — set it there; no longer accepted in [auth] .
session_idle_timeout_minutes			→ moved to [security].sign_out_after_idle_minutes (ADR 0118) — set it there; no longer accepted in [auth] .
session_absolute_hours			→ moved to [security].max_session_hours (ADR 0118) — set it there; no longer accepted in [auth] .
max_sessions_per_user	int	5	cap concurrent sessions per user (ASVS 7.1.2; <code>0</code> = unlimited); a login beyond the cap revokes the user's oldest active session

Key	Type	Default	Notes
<code>step_up_max_age_seconds</code>	int	300	step-up re-verification window (ASVS 7.5.3): a highly sensitive operation requires the session to have re-verified its credential — at login or via <code>POST /me/reauth</code> — within this many seconds. The initial login counts as the first verification (the sudo-timestamp model)
<code>require_action_step_up</code>	bool	<code>true</code>	action-bound step-up (ADR 0077; ASVS 7.5.1/8.2.4). On by default: the durable-takeover JSON routes — TOTP enroll/confirm and disable-MFA — require a fresh proof bound to that specific action (<code>POST /me/reauth</code> with a matching <code>purpose</code> , single-use) instead of riding the session-wide <code>step_up_max_age_seconds</code> window. It closes the most-exploitable default: a session hijacked inside the 300 s login-seeded window could otherwise bind an attacker's authenticator with no fresh proof. It changes only those factor-binding routes — the broad admin / replay / config / purge routes keep the session-window step-up. <code>false</code> reverts to the legacy session-window behaviour (0.2.x semantics), the documented org opt-out
<code>password_min_length</code>	int	15	local-password policy — ASVS 5.0-aligned, length-first
<code>password_require_uppercase</code> / <code>password_require_lowercase</code> / <code>password_require_digit</code> / <code>password_require_symbol</code>	bool	<code>false</code>	character classes — opt-in , each independently (ASVS 5.0 forbids mandatory composition); turn one on only for a legacy standard that still mandates it
<code>password_check_breached</code>	bool	<code>true</code>	reject known common/breached passwords against a bundled offline top-10k list (no live HIBP call)
<code>password_check_context</code>	bool	<code>true</code>	reject passwords containing app/vendor/HL7 terms (e.g. <code>messagefoundry</code> , <code>mefor</code> , <code>hl7</code> , <code>corepoint</code>)
<code>password_check_username</code>	bool	<code>true</code>	reject a password containing the user's own username (ASVS 6.2.11)
<code>password_breach_corpus_file</code>	path	—	optional path to a larger offline breach corpus that augments the bundled top-10k list (ASVS 6.2.12): a plaintext list or an HIBP-style SHA-1 hash export (<code>HASH[:count]</code> lines, auto-detected). Fully offline — still no live HIBP call. Use a curated subset, not the full ~40 GB HIBP set (it is loaded into memory). A path, not a secret
<code>lockout_threshold</code>	int	5	failed logins before lock (per account)
<code>lockout_minutes</code>	int	15	lockout duration
<code>bootstrap_expiry_hours</code>	int	72	the first-run bootstrap admin is auto-disabled once a second administrator exists, and — while still unclaimed (never password-changed) — this many hours after creation. <code>0</code> = no time expiry
<code>bootstrap_warn_hours</code>	int	24	(ASVS 6.4.5) : how long <i>before</i> that deadline to remind an operator that the still-unclaimed bootstrap credential is about to be retired — a <code>bootstrap_admin_expiring [alerts]</code> event, raised once while <code>now</code> is inside <code>[expiry - this, expiry]</code> . Advisory only (it disables nothing); meaningful only when <code>bootstrap_expiry_hours > 0</code> . The deadline itself is also written into <code>bootstrap-admin.txt</code> at issuance.
<code>initial_password_expiry_hours</code>	int	72	(ASVS 6.4.1) : an admin-issued initial/reset credential (a <code>must_change_password</code> temp password) that is never claimed expires this many hours after it was set. Without it an unused reset password grants an authenticated session indefinitely — and the one action it permits is to <i>set the password</i> , i.e. account takeover. Keyed on <code>password_changed_at</code> ; a user who set their own password has <code>must_change_password = false</code> and is unaffected, and the bootstrap admin has its own <code>bootstrap_expiry_hours</code> path (exempt). <code>0</code> = no expiry (not recommended on a PHI instance)

Key	Type	Default	Notes
<code>login_rate_limit_enabled</code>	bool	<code>true</code>	in-process sliding-window limiter on the sign-in surface — <code>/auth/login</code> , <code>/auth/negotiate</code> , <code>/auth/mfa-verify</code> plus the four console entry routes (<code>POST /ui/login</code> , <code>GET /ui/sso</code> , <code>GET /ui/oidc/start</code> , <code>GET /ui/oidc/callback</code>) — in front of the per-account lockout. The same flag also constructs the per-actor credential-ceremony limiter covering <code>/me/password</code> , <code>/me/reauth</code> , <code>/me/mfa/confirm</code> (+ the console re-auth routes); turning it off removes both (see SECURITY.md "Route → limiter map").
<code>login_rate_limit_per_ip</code>	int	10	max attempts per client IP per window (<code>0</code> disables). One number, two limiters : it is also the per- actor budget of the credential- ceremony limiter (<code>/me/password</code> , <code>/me/reauth</code> , <code>/me/mfa/confirm</code> + the console re-auth routes) — the <code>_per_ip</code> name is historical, and retuning it retunes both
<code>login_rate_limit_global</code>	int	60	max attempts across all clients per window (<code>0</code> disables). Sign-in window only — the ceremony limiter has no global dimension (<code>glob=0</code>)
<code>login_rate_limit_window_seconds</code>	float	60	sliding-window length — shared by the sign-in window and the per-actor credential- ceremony limiter, exactly as <code>login_rate_limit_per_ip</code> is
<code>phi_read_rate_limit_enabled</code>	bool	<code>true</code>	per-actor anti-automation throttle (ASVS 2.4.1) — bounds scripted PHI harvesting on top of pagination + access auditing. Charged on 7 JSON routes via <code>require_phi_read</code> , on the 4 bulk-PHI step-up GETs at admission (<code>/messages/search</code> , <code>/messages/export</code> , <code>/uploads/{file_id}/messages</code> , <code>/search/layered</code> — <code>require_step_up</code> paces NON-GET only, so these charge it themselves), and on the 5 /ui PHI views via <code>require_ui(..., phi=True)</code>
<code>phi_read_rate_limit_per_actor</code>	int	120	max PHI reads per user per window (generous — clears console/human use; <code>0</code> disables this dimension)
<code>phi_read_rate_limit_global</code>	int	0	max PHI reads across all users per window (<code>0</code> = off)
<code>phi_read_rate_limit_window_seconds</code>	float	60	sliding-window length
<code>admin_write_rate_limit_enabled</code>	bool	<code>true</code>	per-actor anti-automation pacing on the state-changing admin surface (ASVS 2.4.2) — NON-GET only , charged from one per-actor bucket by both <code>require_step_up</code> and <code>require_paced</code> . JSON API only; no <code>/ui</code> route charges it today (BACKLOG #287)
<code>admin_write_rate_limit_per_actor</code>	int	12	max state-changing admin writes per actor per window (<code>0</code> disables this dimension); there is deliberately no global arm — one operator's bulk work must never throttle another's
<code>admin_write_rate_limit_window_seconds</code>	float	1.0	sliding-window length; over budget → <code>429 + Retry-After: 1</code> , refused before any further work
<code>notify_security_events</code>	bool	<code>true</code>	email the affected user on lockout / first-success-after-failures / password-email-role-disable changes (ASVS 6.3.5/6.3.7). Reuses the <code>[alerts]</code> SMTP transport, sent to the user's own address; no SMTP configured → email skipped. The <code>GET /me/security-events</code> feed (over the audit log) is always available regardless of this toggle. On a PHI production instance this push must be <i>effective</i> — see <code>[alerts].security_notifications_required</code> (BACKLOG #188).
<code>require_mfa</code>			→ moved to <code>[security].require_mfa</code> (ADR 0118) — set it there; no longer accepted in <code>[auth]</code> .
<code>require_mfa_scope</code>			→ set it as <code>[security].require_mfa_scope</code> (ADR 0118) — like its <code>require_mfa</code> sibling it is rejected in <code>[auth]</code> .

Key	Type	Default	Notes
<code>totp_skew_steps</code>	int	<code>0</code>	TOTP clock-skew tolerance in 30 s steps applied at verify time (BACKLOG #187, ASVS 6.5.5). Default <code>0</code> = STRICT: only the current 30 s step verifies (tightest replay window — a captured code is valid at most for the rest of its own step). Set <code>1</code> (or <code>2</code>) — the documented opt-out — to restore RFC-6238 network-delay / clock-drift tolerance (<code>1</code> also accepts the immediately-prior and the fast-clock-clamped next step, i.e. the historical ± 1 behaviour; the forward step is clamped to the current step so it never advances the single-use high-water mark). Range 0–2.
<code>mfa_recovery_code_count</code>	int	10	single-use recovery codes minted at TOTP enrollment (the lost-authenticator escape hatch; <code>0</code> disables them, leaving an admin reset as the only recovery path). Range 0–50.
<code>admin_new_ip_step_up</code>	bool	<code>false</code>	admin-interface contextual-risk signal (WP-L3-13, ASVS 8.4.2): when on, a step-up (sensitive admin) request from a client IP the session has not verified from emits an <code>auth.admin_action_new_ip</code> audit + notice and forces a fresh step-up (a re-verify from that address clears it). Advisory + step-up-forcing only — never changes an RBAC decision, never blocks the non-admin path; the audit + notice fire once per (session, new address). Off by default (byte-identical on loopback — <code>127.0.0.1</code> and <code>::1</code> are treated as one host); recommended on for an off-loopback admin deployment. See SECURITY.md "Administrative-interface defense-in-depth".
<code>ad_enabled</code>	bool	<code>false</code>	turn on Active Directory login
<code>ad_server</code>	str	—	e.g. <code>ldaps://dc1.example.com:636</code> (required when <code>ad_enabled</code>)
<code>ad_domain</code>	str	—	UPN suffix, e.g. <code>example.com</code>
<code>ad_user_search_base</code>	str	—	required when <code>ad_enabled</code>
<code>ad_group_search_base</code>	str	—	base for nested-group resolution
<code>ad_bind_dn</code>	str	—	service-account DN used for lookups
<code>ad_bind_password</code>	secret	—	env only (<code>MEFOR_AUTH_AD_BIND_PASSWORD</code>), or use <code>ad_bind_password_secret</code>
<code>ad_bind_password_secret</code>	str	—	connector <code>SecretProvider</code> reference (ADR 0019 §5) — when set and <code>[secrets].provider</code> is configured, the bind password is resolved from that backend (e.g. a Vault KV <code>path#field</code>) instead of <code>ad_bind_password</code> . A reference, not a secret.
<code>ad_use_nested_groups</code>	bool	<code>true</code>	resolve nested groups (<code>LDAP_MATCHING_RULE_IN_CHAIN</code>)
<code>ad_tls_verify</code>	bool	<code>true</code>	validate the LDAPS certificate
<code>ad_tls_ca_cert_file</code>	str	—	trust an internal CA for LDAPS without disabling verification
<code>ad_tls_ca_cert_pin</code>	str	—	optional lowercase-hex SHA-256 pin over the corresponding CA anchor PEM (<code>ad_tls_ca_cert_file</code>); a mismatch refuses at load + reload (ASVS 6.7.1); unset = no pin (dormant)
<code>ad_allow_insecure_ldap</code>	bool	<code>false</code>	explicit opt-in to a non- <code>ldaps://</code> bind (trusted-network dev only)
<code>ad_connect_timeout</code>	float	<code>10.0</code>	seconds — bounds the LDAP/LDAPS TCP connect on every <code>ldap3 Server</code> the authenticator builds (ASVS 13.1.3). Must be finite and <code>> 0</code> ; <code>0</code> , negative, <code>inf</code> and <code>NaN</code> are refused at config load. <code>ldap3</code> 's own default is <code>None</code> (wait forever), so without this an unresponsive DC pinned a thread-pool worker indefinitely
<code>ad_receive_timeout</code>	float	<code>10.0</code>	seconds — bounds each LDAP response read (both binds and every search) on every <code>ldap3 Connection</code> . Same finite-positive validation

Key	Type	Default	Notes
<code>ad_session_recheck_seconds</code>	int	<code>300</code>	Directory session reconciliation (ADR 0079 mechanism 2). How often to re-resolve directory principals holding live sessions and revoke those AD has disabled or deleted — without it, an AD disable does not take effect until the <code>[security].max_session_hours</code> cap (12 h). 300 (five minutes) is the default (ADR 0148 GIVEN 1 — the hardened path is the shipped path), floored at 60 s (a pass costs one LDAP bind per signed-in directory user). <code>0</code> disables the loop and is a loosening once AD is on — <code>security_loosenings()</code> names it. The default is inert without AD (<code>should_reconcile()</code> also needs an LDAP client), so a non-AD deployment is unaffected; an explicit non-zero value without <code>ad_enabled</code> is still refused rather than left silently dead.
<code>ad_session_recheck_strikes</code>	int	<code>2</code>	Consecutive passes a principal must fail to resolve before its sessions are revoked. The directory lookup collapses <i>disabled</i> , <i>deleted</i> and <i>the search matched nothing</i> into one answer, so a single ambiguous result must never revoke. Range 1–10.
<code>ad_session_recheck_max_users</code>	int	<code>200</code>	Per-pass bind budget. Beyond this, remaining users are picked up by later passes (least-recently-probed first), so a large estate degrades to a longer effective interval instead of a bind storm.
<code>ad_session_revoke_max</code>	int	<code>5</code>	Mass-revoke circuit breaker , absolute half. A bad search base / moved OU / service account that lost read rights answers "not found" for <i>every</i> user — indistinguishable from "everyone was disabled".
<code>ad_session_revoke_max_fraction</code>	float	<code>0.34</code>	Circuit breaker, proportional half. A pass exceeding both thresholds aborts, revokes nothing, logs at ERROR and writes an <code>auth.ad_reconcile_aborted</code> audit row. Requiring both means it fires only on a change simultaneously <i>large</i> and <i>broad</i> — the signature of a misconfiguration, not of offboarding (3-of-3 or 50-of-300 still applies). Range >0.0–1.0; <code>1.0</code> disables the proportional half.
<code>kerberos_enabled</code>	bool	<code>false</code>	Windows SSO (experimental, not yet available ; needs <code>ad_enabled</code>)
<code>kerberos_spn</code>	str	—	service principal, e.g. <code>HTTP/host.example.com</code>
<code>oidc_enabled</code>	bool	<code>false</code>	Federated SSO — OIDC auth-code + PKCE relying party (ADR 0142). A third login for an identity that already exists in on-prem AD (needs <code>ad_enabled</code> ; roles come from LDAP, not the token). Off = byte-identical. Needs <code>[security].web_console_public_address</code> (the redirect origin).
<code>oidc_issuer</code>	str	—	https; exact-matched against the <code>id_token iss</code>
<code>oidc_client_id</code>	str	—	also the required <code>aud / azp</code>
<code>oidc_client_secret</code>	str	—	confidential-client secret — env only (<code>MEFOR_AUTH_OIDC_CLIENT_SECRET</code>), never the file
<code>oidc_client_secret_ref</code>	str	—	alternative: a <code>[secrets].provider</code> reference (<code>_ref</code> , not <code>_secret</code> , to avoid <code>oidc_client_secret_secret</code>)
<code>oidc_authorization_endpoint</code> / <code>oidc_token_endpoint</code> / <code>oidc_jwks_uri</code>	str	—	https, operator-pinned (no <code>.well-known</code> discovery)
<code>oidc_allowed_endpoints</code>	list[str]	<code>[]</code>	defence-in-depth host allow-list; refused empty when enabled ; every OIDC endpoint host must be listed
<code>oidc_tls_ca_cert_file</code>	str	—	the engine's back-channel TLS trust for the IdP (OpenSSL default trust ignores the Windows machine store)
<code>oidc_tls_ca_cert_pin</code>	str	—	optional lowercase-hex SHA-256 pin over the corresponding CA anchor PEM (<code>oidc_tls_ca_cert_file</code>); a mismatch refuses at load + reload (ASVS 6.7.1); unset = no pin (dormant)
<code>oidc_redirect_path</code>	str	<code>/ui/oidc/callback</code>	joined to <code>web_console_public_address</code> for the redirect URI

Key	Type	Default	Notes
<code>oidc_scopes</code>	list[str]	<code>["openid", "profile"]</code>	no <code>email</code> , no <code>offline_access</code>
<code>oidc_signing_algorithms</code>	list[str]	<code>["RS256"]</code>	coerced through the closed JWS algorithm enum
<code>oidc_username_claim</code> / <code>oidc_username_strip_domain</code>	str / bool	<code>preferred_username</code> / <code>true</code>	strip at <code>@</code> → <code>sAMAccountName</code>
<code>oidc_allowed_username_domains</code>	list[str]	<code>[]</code>	the control that stops a federated principal choosing which on-prem account it resolves to. <code>preferred_username</code> is neither unique nor stable (OIDC Core §5.7) and is operator- or even self-editable on many IdPs, so without this a guest presenting <code>Administrator@attacker.example</code> strips to <code>Administrator</code> and signs in as the on-prem Domain Admin. When <code>oidc_username_strip_domain</code> is on, the claim's UPN suffix must match one of these. Empty falls back to <code>ad_domain</code> ; with neither set, <code>oidc_enabled</code> is refused at load rather than stripping unchecked. List the alternate UPN suffixes of a multi-domain forest here
<code>oidc_clock_skew_seconds</code>	int	<code>60</code>	wall-clock skew tolerance (0–300)
<code>oidc_require_mfa_claim</code>	bool	<code>true</code>	#99(g) control — refuse a token with no configured <code>amr</code> / <code>acr</code> . The engine verifies what the IdP asserts , not what it enforced
<code>oidc_mfa_amr_values</code> / <code>oidc_required_acr_values</code>	list[str]	<code>["mfa"]</code> / <code>[]</code>	either family satisfies the gate; both empty with the gate on is refused
<code>oidc_acr_values</code> / <code>oidc_prompt</code>	str	—	requested authorize params
<code>oidc_jwks_ttl_seconds</code> / <code>oidc_jwks_min_refetch_seconds</code>	int	<code>3600</code> / <code>300</code>	the JWKS cache TTL + the amplification (min-refetch) bound
<code>oidc_flow_ttl_seconds</code> / <code>oidc_flow_cache_max</code>	int	<code>300</code> / <code>512</code>	pending-flow TTL + the reject-when-full bound
<code>oidc_session_max_hours</code>	int	—	caps the federated session below <code>id_token.exp</code> if a tighter bound is wanted (ADR 0079 mechanism 1)

AD-group→role mappings live in the DB and are managed by an admin (`PUT /ad-group-map` or the console Users page), not in this file. Federated logins reuse the **same** AD-group→role mapping — the role source is on-prem AD, never a token claim (ADR 0142).

[ai] — AI coding assistance policy

Implemented (see [AI.md](#)). Controls the IDE AI assistant across the **OFF→PHI-safe** range; the policy is centrally governed and **posture-clamped**. `mode` / `data_scope` plus the active environment NAME + posture (`environment` / `data_class` / `production`) govern the policy; `provider`, `model`, `endpoint`, `api_key` and `allowed_endpoints` are **live** under `mode = managed_endpoint` — the engine broker (ADR 0135). Only `baa_attested` is still a forward-compat placeholder (accepted-but-ignored).

Key	Type	Default	Notes
<code>mode</code>	enum	<code>byo</code>	<code>off</code> · <code>byo</code> · <code>managed_endpoint</code> · <code>managed_claude</code> · <code>managed_claude_baa</code> . <code>managed_endpoint</code> is built — the engine brokers one <code>code_only</code> prompt to a customer-managed / self-hosted LLM over <code>POST /ai/chat</code> , audited per use (ADR 0135); it never reaches <code>phi</code> scope. <code>managed_claude</code> / <code>managed_claude_baa</code> are future — not serviceable by the current IDE
<code>data_scope</code>	enum	<code>code_only</code>	<code>code_only</code> · <code>synthetic</code> · <code>deidentified</code> · <code>phi</code> , least→most sensitive; capped by <code>production</code> posture and by <code>mode</code> (only <code>managed_claude_baa</code> reaches <code>phi</code>)
<code>environment</code>	str	—	free-form active-environment name (ADR 0017); selects <code>environments/<name>.toml</code> + <code>current_environment()</code> . Required for <code>serve</code> (no default)
<code>data_class</code>			→ moved to <code>[security].handles_real_patient_data</code> (ADR 0118) — set it there; no longer accepted in <code>[ai]</code> .

Key	Type	Default	Notes
<code>production</code>			→ moved to <code>[security].production_instance</code> (ADR 0118) — set it there; no longer accepted in <code>[ai]</code> .
<code>provider</code>	str	<code>claude</code>	the broker's request shape; read under <code>mode = managed_endpoint</code> (and recorded in the per-use audit)
<code>model</code>	str	<code>claude-opus-4-8</code>	the model the broker asks for; read under <code>mode = managed_endpoint</code> (also echoed on the reply)
<code>baa_attested</code>	bool	<code>false</code>	accepted-but-ignored — an operator attestation carried for the future <code>managed_claude_baa</code> path
<code>endpoint</code>	str	—	the customer-managed LLM URL. Required for <code>mode = managed_endpoint</code> ; <code>http / https</code> only, and a cleartext <code>http</code> endpoint is refused (it would expose <code>api_key</code>)
<code>api_key</code>	secret	—	the LLM credential — env only (<code>MEFOR_AI_API_KEY</code>), never the file. Required for <code>mode = managed_endpoint</code>
<code>allowed_endpoints</code>	list	<code>[]</code>	fail-closed SSRF allow-list for <code>endpoint</code> 's host; each entry is <code>host</code> (any port) or <code>host:port</code> . An empty list permits nothing — deliberately independent of <code>[egress].allowed_http</code> , which is permissive when empty and so can't gate this surface

Only `code_only` context is ever sent in the MVP (graph names + active editor code) — **never message bodies**. The full resolution/clamping algorithm, the `GET /ai/policy` endpoint, the `messagefoundry ai-policy` CLI, and the `ai:assist` RBAC permission are documented in `AI.md`. Env keys: `MEFOR_AI_MODE`, `MEFOR_AI_DATA_SCOPE`, `MEFOR_AI_ENVIRONMENT`, etc.

[logging]

Key	Type	Default	Notes
<code>level</code>	enum	<code>info</code>	log level; never run prod at <code>debug</code> (PHI) — <code>serve</code> refuses it (Gate #1)
<code>format</code>	enum	<code>text</code>	stdout rendering: <code>text</code> (default) or structured <code>json</code> (one object per line). Stdlib only — no structlog
<code>log_dir</code>	str	<code>unset</code>	the directory NSSM (or another supervisor) rotates the engine's captured stdout/stderr into . The engine never writes log files itself (it logs to stdout); set this only to tell it where the supervisor parks them, and <code>GET /status</code> then meters that directory's total bytes + filesystem free space alongside the DB metrics (#50). Unset = stdout-only, no metering. Metadata only — the file contents are never read.
<code>forward_enabled</code>	bool	<code>derived</code>	ship a copy of every record off-box to a syslog/SIEM collector (sec-offbox-log) so evidence survives a host compromise. Default-on-when-configured (ADR 0080) : <code>unset</code> ⇒ on iff <code>forward_host</code> is set. Set <code>false</code> to opt out even with a host; no <code>forward_host</code> ⇒ off (stdout-only, unchanged)
<code>forward_host</code>	str	—	syslog/SIEM collector host. Setting it turns forwarding on by default (above)
<code>forward_port</code>	int	<code>514</code>	collector port (1–65535)
<code>forward_protocol</code>	enum	<code>udp</code>	<code>udp</code> (fire-and-forget), <code>tcp</code> , or <code>tls</code> (RFC 5425 — native <code>ssl</code> -wrapped TCP, ADR 0080). A <code>tcp / tls</code> collector down at startup is skipped with a warning; a runtime stall is bounded by a socket timeout (record dropped) and the TLS handshake is bounded too, so a wedged collector never blocks the engine. Synchronous send — prefer <code>udp</code> / a local agent for high volume
<code>forward_format</code>	enum	<code>json</code>	wire format sent off-box, independent of stdout <code>format</code> . JSON guarantees one record per line; <code>text</code> framing is best-effort (multi-line tracebacks span lines)
<code>forward_tls_ca_file</code>	str	—	PEM trust anchor for the collector's cert (required when <code>forward_protocol = "tls"</code> and verification is on). Only this CA is trusted — the public system bundle is not loaded, so an on-prem SIEM's private cert is anchored explicitly
<code>forward_tls_verify</code>	bool	<code>true</code>	verify + hostname-check the collector's certificate. <code>false</code> is the documented insecure opt-out (<code>CERT_NONE</code> , no CA file needed) — lab / pinned-network only
<code>forward_tls_client_cert</code>	str	—	optional PEM cert+key chain for mutual TLS to the collector

Key	Type	Default	Notes
<code>forward_hop_attested</code>	bool	<code>false</code>	acknowledged opt-out for a plaintext / unverified-TLS collector hop (#200, ADR 0092 — the <code>[logging]</code> sibling of a connection's <code>tls_hop_attested</code>). A hop that is not verified TLS is now decided by the shared posture gradient: refused on an enforcing PHI instance, warned on a non-enforcing PHI instance, allowed for loopback / synthetic. Set this (with a reason) to affirm the hop is secure by other means — e.g. a dedicated out-of-band management VLAN
<code>forward_hop_attested_reason</code>	str	—	why the hop is secure, recorded for the audit trail. Mandatory when <code>forward_hop_attested = true</code> (ADR 0153 retro-fitted the flag-implies-reason rule: an attestation that suppresses a refusal must record WHY, or it is worthless when audited) — the flag alone now fails at load. Rejected without the flag, and must be non-empty
<code>require_time_sync</code>	bool	<code>false</code>	opt-in startup clock-sync gate (ASVS 16.2.2, ADR 0080): before listeners start, probe <code>ntp_peer</code> and warn on skew. Requires <code>ntp_peer</code> . Default = no-op
<code>ntp_peer</code>	str	—	NTP/SNTP host to compare the local clock against (required when <code>require_time_sync</code>)
<code>time_sync_max_skew_seconds</code>	float	<code>2.0</code>	$ \text{local} - \text{peer} $ above this is "skewed" (must be > 0)
<code>time_sync_fail_closed</code>	bool	<code>false</code>	refuse to start (instead of warn) on skew or an unreachable peer. Further opt-in; requires <code>require_time_sync</code>
<code>file</code> , <code>max_bytes</code> , <code>backups</code>	str/int	—	accepted-but-ignored (planned) rotation — none is a <code>LoggingSettings</code> field. The engine logs to stdout and NSSM rotates it; <code>log_dir</code> above is how you point the engine at where it lands

PHI redaction + control-char scrubbing are **always-on handler filters** (not a toggle) applied to **every** sink, including the off-box forwarder. For an encrypted hop set `forward_protocol = "tls"` (native RFC 5425, no agent needed); alternatively front a plaintext `udp / tcp` forward with a local TLS-forwarding agent or a trusted network. See [PHI.md §7](#) and [ADR 0080](#).

The plaintext default is now gated, not silent (#200, ADR 0092). The forwarded stream is PHI-redacted but still carries usernames, connection names, message ids, client addresses, and the tamper-evident audit chain. `serve` therefore decides the forwarding hop with the **same** authority the transports use, *before* the handler is installed: a hop that is not verified TLS is **refused** on an enforcing PHI instance, **warned** on a non-enforcing PHI one, and **allowed** for a loopback collector (so the "plaintext to `127.0.0.1` + a local agent" deployment is untouched) or a synthetic instance. To keep a plaintext off-box hop, either move to `forward_protocol = "tls"` or set `forward_hop_attested` with a reason — an acknowledged escape, not a silent default.

[retention]

Enforced by the engine's retention/purge task ([pipeline/retention.py](#)). A purge **NULLS the PHI body** past its window while **keeping the message row** (counts, disposition, and the audit trail stay intact — the Mirth Data-Pruner pattern); it never deletes a `messages` row and never touches a body still in flight. The *row* survives; its PHI *columns* do not — `messages.metadata` is nulled in the same statement as the body (ASVS 14.2.7). The raw `[retention]` fields still default to `0 / ""` = keep/off, **but** `serve` **applies a posture gate on top of them, so retention is not opt-in on a PHI instance:** under `[security].enforcement = enforce` (the default) an unbounded `[security].delete_message_bodies_after_days` or `[retention].dead_letter_days` **refuses to start (exit 2)**; on a non-enforcing PHI instance each *unset* window is auto-bounded to **30 days**. All three built-in environment names (`dev`, `staging`, `prod`) derive PHI. The audited opt-out is `[security].allow_keeping_phi_indefinitely = true`. See [PHI.md §8](#).

Key	Type	Default	Notes
<code>messages_days</code>			→ moved to <code>[security].delete_message_bodies_after_days</code> (ADR 0118) — set it there; no longer accepted in <code>[retention]</code> .
<code>dead_letter_days</code>	int	<code>0</code>	past N days, null the bodies of dead-lettered outbound rows (their own window — a dead row stays replayable until purged). <code>0</code> = keep
<code>allow_unbounded_phi</code>			→ moved to <code>[security].allow_keeping_phi_indefinitely</code> (ADR 0118) — set it there; no longer accepted in <code>[retention]</code> .

Key	Type	Default	Notes
state_max_age_days	int	0	past N days, delete transform-state entries (ADR 0005) last written before the cutoff — keeps the in-memory state cache + table bounded. A simple global age purge (by <code>set_at</code>); per-namespace policy is a follow-up. 0 = keep
connection_event_retention_hours	int	0	past N hours , delete <code>connection_event</code> rows (the <code>[diagnostics]</code> #46 transport/lifecycle log — high-volume under a connect-per-message sender or a probe storm, so its own short window in hours , not days). 0 = inherit the <code>messages_days</code> body window (the ADR 0021 §7.5 default).
app_log_days	int	0	past N days, delete application log files (<code>.log / .txt</code> , one level) from the configured <code>[logging].log_dir</code> (#120). The supervisor (NSSM <code>AppRotateBytes</code>) rotates the daily logs by size but never by age , so the log dir grows unbounded; this bounds it (by file mtime, so the currently-written file is never eligible). 0 = keep. No-op unless <code>[logging].log_dir</code> is set . Metadata only — file content is never read. While <code>app_log_compress_days</code> is on, the same window also ages out the <code>*.log.gz / *.txt.gz</code> archives that setting produces — so compressing a log doesn't make it immortal; with compression off the eligible set is exactly what it was
app_log_compress_days	int	0	past N days, gzip application log files (<code>.log / .txt</code> , one level — the same selection as <code>app_log_days</code> , by mtime, so the currently-written file is never eligible) in <code>[logging].log_dir</code> to <code><name>.gz</code> (#119). The log stays readable (<code>gzip -d</code>) at a fraction of the disk, so a long-running box keeps far more history for the same footprint. Each file is free-space prechecked (<code>shutil.disk_usage</code> must show room for the source plus its archive plus a <code>max(10%, 1 MiB)</code> margin — short, and the file is skipped and logged , never attempted) and each written archive is integrity-validated — staged to an exclusively created, randomly named temp file beside it (<code>tempfile.mkstemp: O_CREAT O_EXCL</code> , so it never truncates an existing file, never follows a symlink, and never collides with a sibling engine shard compressing the same directory), <code>fsync</code> ed, re-read off disk , decompressed and compared byte-for-byte against the original, renamed into place, and then validated again at <code><name>.gz</code> itself — and it is that last check, on the bytes actually sitting where the log used to be, that authorizes removing the original. Any failure leaves the original in place , does not count it as compressed, and logs it; an existing <code><name>.gz</code> is never clobbered. The archive inherits the source's mtime, so <code>app_log_days</code> still ages it out. Files over 64 MiB are skipped (the codec is in-memory), and so is a file whose archive would not be smaller than it (an empty or already-compressed log — compressing must never <i>cost</i> disk). Names/counts/sizes are logged, never file content . 0 = never compress. No-op unless <code>[logging].log_dir</code> is set . Set it shorter than <code>app_log_days</code> — a longer window compresses nothing, since the delete sweep runs first
search_preset_days	int	0	past N days, delete saved-search presets (ADR 0136) neither used nor edited since the cutoff. The stored <code>criteria</code> is the operator's own content/ <code>field_value</code> needle — PHI-shaped by construction , encrypted at rest — so it needs a window like any other PHI tier (ASVS 14.2.7). The whole row is deleted, not blanked: a preset's entire payload <i>is</i> its criteria. Keys on last-USED (BACKLOG #306) — the cutoff is compared against the <i>later</i> of <code>updated_at</code> (written by a save) and <code>last_used_at</code> (written by a recall), so a preset you run daily but never re-save is kept . A preset last touched before the <code>last_used_at</code> column existed has it NULL and ages out on <code>updated_at</code> alone. 0 = keep forever (the default)
audit_days	int	0	reserved / not enforced . The <code>audit_log</code> is a tamper-evident hash chain and HIPAA expects ~6-year retention, so audit is keep-forever by design ; archive-first pruning is a tracked follow-up. Accepted so a forward-looking file still loads
max_db_mb	int	0	advisory only: warn (WARNING log + an <code>AlertSink storage_threshold</code> event) when the database exceeds this — measured as the <code>SQLite file + -wal / -shm</code> , <code>SUM(size)</code> over <code>sys.database_files</code> on SQL Server , and <code>pg_database_size()</code> on Postgres . Never auto-deletes. 0 = off
purge_interval_seconds	float	3600	how often the purge/maintenance loop runs a pass
max_pass_seconds	float	0	maximum wall-clock seconds one maintenance pass may spend (#121, ADR 0137). A between-phase soft cap : <code>run_once</code> checks elapsed monotonic time before each phase and, once this is reached, skips the remaining phases (marking the pass <code>capped</code>) so a long pass can't run unbounded into the next maintenance window — the skipped tail re-runs next interval, and a skipped WAL-checkpoint/VACUUM does not advance its last-run marker. Checked only <i>between</i> phases, never inside one, so a running VACUUM is non-interruptible. 0 = off (the default — no cap); ~ 14400 (4 h, the Corepoint off-peak ceiling) is the recommended value when enabled

Key	Type	Default	Notes
wal_checkpoint_seconds	float	0	PRAGMA wal_checkpoint(TRUNCATE) cadence — SQLite only; a documented no-op on SQL Server and Postgres , where log management is a DBA operation. 0 = off (rely on auto-checkpoint). Evaluated once per pass, so a value below purge_interval_seconds is effectively rounded up to it
vacuum_at	str	""	daily local "HH:MM" to run VACUUM (reclaims space freed by purges) — SQLite only; a documented no-op on SQL Server and Postgres , where space reclamation is a DBA operation. "" = off. A daily off-peak time, not a cron expression (no new dependency); VACUUM holds a write lock on the whole DB while it runs

Per-connection overrides (ADR 0027). messages_days and dead_letter_days are **global defaults** an individual connection may override: an **inbound** sets its own messages_days, an **outbound** its own dead_letter_days (both None = inherit this global window, 0 = keep that connection's bodies forever, >0 = days). An inbound may also opt into **embedded-document pruning** (prune_documents_after + prune_documents_min_bytes, ADR 0042) to strip bulky base64 attachments while keeping the readable message. These live on the connection (code-first or in connections.toml) — see CONNECTIONS.md.

Backend coverage. The retention/purge pass is **backend-agnostic** and every PHI purge runs on **all three** backends (SQLite, SQL Server, Postgres) — pipeline/retention.py contains no backend branch. Only wal_checkpoint_seconds and vacuum_at are SQLite-only: on the server backends those methods are documented no-ops, and log management / space reclamation (plus the DB-tier [backup] snapshot) are DBA operations there. Each pass that does real work writes one retention_purge audit_log entry (cutoffs + counts, **no** message content). Per-backend table: PHI.md §8.

[update_check]

Engine-side version-update check (ADR 0026). The MVP is a **no-network "pinned-vs-current" diff**: it compares the running messagefoundry.__version__ against the version in the installed distribution metadata (importlib.metadata) / the bundled requirements.lock — **zero outbound traffic**. The result is one additive /status field and (optionally) one update_available AlertSink event that the console/IDE render as a dismissible banner — **the console/IDE never call PyPI**. Because the local diff is cheap and PHI-safe it is **on by default** (zero phone-home risk).

Key	Type	Default	Notes
enabled	bool	true	emit the /status field + the update_available alert. false = suppress both
check_interval_seconds	float	86400	diff cadence (the diff is trivial; daily is ample). Must be > 0
mode	str	"local"	the no-network diff (the only MVP value). "live" (the constrained-egress path, ADR 0026 §2) is defined but rejected at load , so a config can never silently turn the check into a phone-home out of a PHI system
index_url , index_allowed_hosts	str / list	unset	forward-compat for the future "live" mode only — accepted-but-unused in the MVP

[delivery]

Key	Type	Default	Notes
retry_max_attempts	int	unset	attempts before a delivery dead-letters. Unset = retry forever (the conservative default — a transient failure/ AE NAK is never silently lost; under FIFO the head blocks its lane until it succeeds or is purged). Set a finite value to opt back into retry-then-dead-letter. A permanent AR reject fails fast regardless.
retry_backoff_seconds , retry_backoff_multiplier , retry_max_backoff_seconds	num	5 / 2 / 300	exponential backoff between attempts (per-outbound retry= overrides)
ordering	enum	fifo	default queue ordering per outbound: fifo (strict in-order, head-of-line on failure) or unordered (batch + rotate-past-failures). Per-outbound ordering= overrides.

Key	Type	Default	Notes
<code>internal_error</code>	enum	<code>continue</code>	what a delivery worker does on an internal/code error (a non- <code>DeliveryError</code> exception from <code>send</code> — our bug, not the partner's): <code>continue</code> (dead-letter the row + advance) or <code>stop</code> (halt the connection's worker, preserve the message for replay, raise a <code>connection_stopped</code> alert). Per-outbound <code>internal_error=</code> overrides. Partner NAKs / transport failures are unaffected.
<code>buildup_max_depth</code>	int	<code>unset</code>	raise a <code>queue_buildup</code> alert when an outbound lane's pending depth reaches this. Unset = depth dimension off (a healthy ceiling is throughput-specific, so there's no safe default). Per-outbound <code>buildup=BuildupThreshold(...)</code> overrides.
<code>buildup_max_oldest_seconds</code>	num	300	raise <code>queue_buildup</code> when the lane's oldest pending message has waited this long (a stuck/retry-forever head is the classic cause). On by default — a head stuck > 5 min is a problem in any environment. Set to unset/ <code>0</code> -disable via a per-outbound override.
<code>stall_max_oldest_seconds</code>	num	<code>unset</code>	raise a <code>message_stall</code> alert (Corepoint "Max Message Stall", ADR 0014) when an outbound lane's oldest undelivered message has waited this long. Unset (the default) = the stall alert is OFF — deny-by-default/opt-in, because it overlaps <code>buildup_max_oldest_seconds</code> 's age dimension and would double-page if both fired. Set a threshold to turn it on; a per-outbound <code>stall=StallThreshold(...)</code> overrides it. The stall event routes through <code>[[alerts.rules]]</code> like any other (ADR 0014).
<code>saturation_sustain_samples</code>	int	<code>unset</code>	raise a <code>saturation</code> alert (BACKLOG #93, ADR 0014 amendment) when an outbound lane's pending backlog is rising sustained over this many consecutive samples — the queue derivative (ingest > drain), distinct from the absolute depth/age ceilings above. A bursty-but- draining lane (spike then fall) never fires; only a lane whose depth climbs monotonically does. Unset (the default) = OFF — deny-by-default/opt-in (it overlaps <code>buildup_max_oldest_seconds</code> 's age dimension). Floor of 2 (fewer can't tell a burst from sustained growth). Global-only for now; a per-outbound override is a documented follow-up (a <code>[[alerts.rules]] connection glob with transports = []</code> can suppress it for a known-bursty feed in the interim).
<code>priority</code>	enum	<code>normal</code>	global DR / priority tier default for every connection (#61, ADR 0048). A connection declaring no <code>priority=</code> of its own inherits this; resolution order is per-connection override > this global default > the built-in <code>normal</code> . The total order is <code>critical</code> > <code>normal</code> > <code>low</code> , and the <code>[dr]</code> run-profile starts only connections whose resolved rank is at or above <code>[dr].priority_threshold</code> . It governs when a connection runs , never what it does. <code>normal</code> keeps every connection at the same tier, so a deployment that never enables DR is byte-unchanged. An unknown value fails config load.
<code>outbox_workers</code>	int	—	accepted-but-ignored (planned): delivery concurrency. Not a <code>DeliverySettings</code> field — worker topology is set by <code>[pipeline].claim_mode</code> today
<code>dead_letter</code>	enum	—	accepted-but-ignored (planned): <code>keep</code> / <code>drop</code> -after-N. Not a <code>DeliverySettings</code> field — a finite <code>retry_max_attempts</code> is what dead-letters a row today

[pipeline]

Key	Type	Default	Notes
<code>max_correlation_depth</code>	int (≥1)	8	Re-ingress loop cap (ADR 0013 Increment 2). When a captured reply is re-ingressed (<code>reingress_to= / Loopback()</code>), the re-ingressed message carries a <code>correlation_depth</code> ; a message at this depth still routes, but the next hop (depth+1) dead-letters its re-ingress work-row and marks the origin <code>ERROR</code> . Coarse by design — it bounds <i>total work</i> , not topology, so a chain that legitimately bounces A→B→A a few times needs headroom. 8 is safe for typical request→response→route feeds; raise it for deep correlation chains, lower it to fence a misbehaving loop. (A value of 0 would dead-letter every re-ingress, so the floor is 1.)

Key	Type	Default	Notes
<code>per_lane_wake</code>	bool	<code>false</code>	Per-lane wake events (B12, ADR 0061). Reliability-core, default-OFF. When <code>false</code> , a committed message wakes every worker of its stage via an engine-wide event (the historical behavior). When <code>true</code> , it wakes only its own (stage, lane) worker , eliminating the thundering-herd empty-claim storm that dominates at high connection counts (~1,500 inbounds). Correctness is unchanged (the FIFO claim + the 0.25 s lost-wakeup poll backstop are untouched; a missed wake self-heals within the poll). Read once at engine start — a <code>/config/reload</code> does NOT toggle it (restart to change). Env override (for the connection-scale harness A/B): <code>MEFOR_PIPELINE_PER_LANE_WAKE=true</code> . Applies only in <code>per_lane</code> claim mode (see <code>claim_mode</code>); the default <code>pooled</code> mode routes wakes through its dispatchers instead, so this knob is inert there.
<code>claim_mode</code>	enum	<code>pooled</code>	Pipeline claim mode (ADR 0066). Reliability-core. <code>pooled</code> (the default since #744) runs one <code>StageDispatcher</code> per stage — a handful of shared claimer tasks batch-claim head-prefixes across lanes, collapsing the per-connection claim storm and holding zero-loss at high fan-out where <code>per_lane</code> drops messages. <code>per_lane</code> is the byte-identical opt-out (<code>[pipeline].claim_mode = "per_lane"</code>): the pre-ADR-0066 topology of one router+transform worker per inbound and one delivery worker per outbound, enforced by a test sentinel. Read once at engine start — a <code>/config/reload</code> does NOT toggle it (restart to change). Env override (harness A/B): <code>MEFOR_PIPELINE_CLAIM_MODE</code> . Two caveats (see <code>CONNECTIONS.md</code> "Pipeline claim mode"): exactly-once degrades under load (no inbound de-dup — receivers must be idempotent; not pooled-specific) and active-passive failover-under-load is covered (the gated <code>test_load_failover_{postgres,sqlserver}</code> two-node kill-the-leader runs hold no-acknowledged-loss / per-lane FIFO / bounded dup-rate under pooled; only recovery <i>time</i> is host-dependent, and the T17 infra-fault spin is bounded by ADR 0070). Invariants (at-least-once / per-lane FIFO / poison-guard) are unchanged in both modes.
<code>pooled_claimers_per_stage</code>	int (≥1)	1	Pooled-only: K claimer tasks per stage (>1 hash-partitions lanes across claimers so no two claim the same lane).
<code>pooled_sweep_interval</code>	float (>0)	0.25	Pooled-only: the clock-driven discovery-sweep interval (the bounded at-least-once backstop; 0.25 s = <code>poll_interval</code> parity).
<code>pooled_claim_lane_chunk</code>	int (1–500)	256	Pooled-only: max lanes batch-claimed per claim round-trip (clamped down to the backend store's chunk — SQLite 200, SS/PG 500).
<code>pooled_max_processing_lanes</code>	int (≥1)	256	Pooled-only: max concurrently-processing lanes per stage (the decrypted-body / crash-exposure bound).
<code>require_rcsi_for_pooled</code>	bool	<code>true</code>	Pooled-only (SQL Server): fail closed at startup if <code>READ_COMMITTED_SNAPSHOT</code> is OFF (the §3.2 correctness proofs assume RCSI on); <code>false</code> downgrades to a loud warning + a <code>/stats rcsi_off_degraded</code> gauge. No-op on SQLite/Postgres.
<code>infra_fault_policy</code>	enum	<code>stop</code>	Pooled T17 (infra/machinery-fault) handling (ADR 0070). A store/handoff error — or any raise from outside the per-item body — is caught by the dispatcher's T17 handler, which always re-pends the faulting head at an exponential-capped backoff (collapsing the ~4×/s sweep spin). This key bounds a persistent such fault: <code>stop</code> (default) STOPs the head-of-line-blocked lane after <code>infra_fault_stop_after</code> consecutive zero-progress faults, reusing the <code>internal_error = stop</code> muscle (STOPPED phase + <code>connection_stopped</code> alert; reload / new work re-arms) and never dead-lettering the good message. <code>retry_forever</code> never STOPs — it retries the head at capped backoff forever and emits a throttled <code>lane_stuck</code> alert once the horizon is crossed (for a deliberately-unattended flaky-infra site). Reliability-core: read once at engine construction — a <code>/config/reload</code> does NOT re-read it (restart to change).
<code>infra_fault_stop_after</code>	int (≥1)	10	consecutive zero-progress T17 faults before a <code>stop</code> -policy lane transitions to STOPPED — and the stuck horizon at which <code>retry_forever</code> 's throttled <code>lane_stuck</code> alert first fires. Under the exponential backoff (capped by <code>infra_fault_backoff_cap</code>) 10 spans ~4 min of wall clock, so it is really a duration gate.
<code>infra_fault_backoff_cap</code>	float (>0)	60.0	cap (seconds) on the T17 head re-pend backoff — base is the dispatcher's 1 s lane-error backoff, doubling per consecutive zero-progress fault. ~60 s picks a recovered dependency back up within about a minute while still collapsing the spin.

Key	Type	Default	Notes
<code>fuse_thread_hops</code>	bool	<code>false</code>	Thread-hop fusion (ADR 0071 B5). Reliability-core, default-OFF, SQL-Server-scoped : when <code>true</code> and the store backend is SQL Server and <code>claim_mode = "pooled"</code> , each fused stage (INGRESS/ROUTED) runs its off-loop CPU stage (<code>route_only</code> / <code>transform_one</code>) together with its store handoff on a single dedicated-executor worker hop, collapsing a multi-statement aiiodbc handoff into one executor→loop completion (the profiled per-completion async-marshaling wall). Provably no-op elsewhere: Postgres (asynccpg is loop-native) and SQLite (loop-affine handoff lock) keep the async path by construction and log "ignored", and a sync-handoff-pool open failure downgrades to the async path with a loud warning + a degraded gauge — never a lane outage. Read once at engine construction (restart to change) . Env/harness A/B: <code>MEFOR_PIPELINE_FUSE_THREAD_HOPS</code> .
<code>pooled_fusing_workers</code>	int (≥1)	8	worker count for each per-stage fusing executor (ADR 0071 B5). Every fused stage gets its own <code>ThreadPoolExecutor</code> of this width plus a matching-width dedicated synchronous pyodbc handoff pool (one connection per worker, so a fused hop never blocks acquiring). Small by default — a fused hop holds a worker across DB latency, so this is the fused-stage concurrency; it also clamps the fused stages' effective <code>pooled_max_processing_lanes</code> to ~2× this value, so the claimer doesn't reserve 256 slots for a handful of workers. Inert unless <code>fuse_thread_hops</code> is on.
<code>batch_handoff_statements</code>	bool	<code>true</code>	Per-hop SQL statement batching (ADR 0075). Default-ON (promoted 2026-07-08; retained as an emergency off-switch) and SQL-Server-scoped: each per-hop staged handoff (<code>route_handoff</code> / <code>transform_handoff</code>) folds the non-result-returning DML of its body into the fewest <code>pyodbc.execute()</code> T-SQL batches — the same ordered (<code>sql</code> , <code>params</code>) sequence, one round-trip per batch, still committing exactly once per hop . It cuts network round-trips, not transactions: no commit boundary moves, the claim stays its own poison-guard transaction, and the ACK-on-receipt fence is untouched. Each result-consuming statement whose value gates later control flow (the guard <code>DELETE</code> , the finalize <code>GROUP BY</code> , the finalize <code>sp_getapplock</code> rc-check) stays its own execute. Postgres and SQLite have no batched path and run byte-identically. Read once at engine construction (restart to change) . Env: <code>MEFOR_PIPELINE_BATCH_HANDOFF_STATEMENTS</code> .
<code>snapshot_on_send</code>	bool	<code>true</code>	Copy-on- Send (ADR 0104) — snapshot each <code>Send</code> 's payload at construction so a divergent fan-out (mutate between <code>Send</code> s) delivers per-destination state instead of a last-write collapse. Default-ON since the BACKLOG #230 default-flip: the conservative estate AST scan flagged 1 of 152 handlers and human triage found that one mutates an independent clone, so genuine divergence is 0 and the flip changes delivered bytes for no handler; and <code>Message.copy()</code> is now genuine copy-on-write, so the common single- <code>Send</code> / no-post-mutation path is zero-copy and a deepcopy fires only on an actual divergence. Set <code>false</code> to restore the pre-ADR-0104 last-write behaviour. Backend-agnostic. Read once at engine construction (restart to change) . Env: <code>MEFOR_PIPELINE_SNAPSHOT_ON_SEND</code> .
<code>credential_fault_policy</code>	enum	<code>stop</code>	Partner-account-lockout protection (#109, ADR 0095). What an outbound File/FTP/SFTP sender does on a permanent credential/auth fault (bad password, key rejected). <code>stop</code> (default) halts the lane immediately (not after a streak) and retains the queued rows un-errored (they stay pending/claimable, never dead-lettered), so a backlog can't re-authenticate in a loop and trip the partner's account lockout — reusing the STOP muscle (<code>connection_stopped</code> alert; reload/restart re-arms the lane once the credential is fixed). <code>dead_letter</code> keeps the historical fail-fast (dead-letter just the offending row and advance). A content -permanent reject (AR/CR, no-such-dir) is unaffected — it still dead-letters.
<code>schedule_tick_seconds</code>	float (>0)	30.0	Active-window scheduler tick (#147, ADR 0095). The reconcile granularity for a connection's per-connection <code>schedule</code> (a window boundary is honoured within one tick). Only affects connections that declare a <code>schedule</code> ; connections with none are byte-identical always-on.

[sandbox]

Opt-in subprocess isolation for Routers/Handlers (ADR 0087, BACKLOG #197, ASVS 15.2.5). Routers/Handlers are admin-authored pure Python the engine runs in its own address space (the DEK, audit chain, and live sockets live there). `mode="off"` (the default) runs them in-process, **byte-identically and with zero overhead**. `mode="subprocess"` runs each inbound's Router/Handler in a **persistent per-inbound worker child** (never a per-message fork), enforcing a forbidden-import guard (socket/store/crypto), the resource caps below, and a **fail-closed** refusal of the live `db_lookup` / `fhir_lookup` bridges (they re-enter the event loop — a subprocess boundary breaks that; a Handler

needing live enrichment runs with `mode=off`). An isolation denial (forbidden op, cap overrun, worker crash) routes the message to `ERROR` /dead-letter **post-ACK** (no NAK, never dropped). **Read once at engine start — a `/config/reload` does NOT re-read it (restart to change).**

Key	Type	Default	Notes
<code>mode</code>	enum	<code>off</code>	<code>off</code> (in-process, byte-identical, no subprocess) or <code>subprocess</code> (persistent per-inbound worker child).
<code>wall_seconds</code>	float (>0)	5.0	Authoritative wall-clock cap per Router/Handler call on every platform — the parent kills a worker that overruns it, so a pathological busy-loop can't wedge intake.
<code>cpu_seconds</code>	float (>0)	2.0	POSIX-only <code>RLIMIT_CPU</code> backstop inside the child (a no-op on Windows, where <code>wall_seconds</code> governs).
<code>mem_mb</code>	int (≥1) or null	512	POSIX-only <code>RLIMIT_AS</code> address-space cap (MiB) inside the child (no-op on Windows). <code>null</code> disables it.
<code>startup_seconds</code>	float (>0)	30.0	Bound on the one-time child bootstrap (config load + guard install) before start fails closed.

[diagnostics]

The Corepoint-style **event log** (#46) — a metadata-only record of connection lifecycle / pre-ingress failures and the ACK/NAK the engine returns. **Both master switches are on by default and safe to be:** they store only non-PHI metadata (connection name, peer IP, a scrubbed reason, the ACK disposition), and the AA-ACK *body* is stored only when the store is encrypted (else NULL); a NAK body is never persisted. A per-connection `capture_connection_errors` / `capture_ack` flag overrides the matching master switch for one connection (see [CONNECTIONS.md](#)). `message_events`, the third row below, is **not** a master switch but a **verbosity dial** over the per-message disposition log — it defaults to `all` and keeps a compliance floor even at `off`. The fourth row is a relocated key, rejected here since ADR 0118.

Key	Type	Default	Notes
<code>connection_events</code>	bool	<code>true</code>	master switch for the connection/transport event log : inbound lifecycle (established/closed) + pre-ingress failures (allowlist/capacity/oversize/peer-reset/framing) + outbound lane transitions (connection_lost/restored). Metadata-only, written off the hot path by a drain task. Per-connection <code>capture_connection_errors</code> overrides it.
<code>response_sent</code>	bool	<code>true</code>	master switch for "Response Sent" — the ACK/NAK returned to an inbound sender. Always captures the disposition metadata (<code>ack_code</code> / <code>phase</code> / <code>outcome</code>); the AA body is stored only on an encrypted store, and every NAK body is NULL. Per-connection <code>capture_ack</code> overrides it.
<code>message_events</code>	enum	<code>all</code>	verbosity of the per-message <code>message_events</code> disposition log (#63) — how many rows the store writes to that table. <code>all</code> (default) records every event; <code>errors</code> drops the routine successes (<code>received</code> / <code>delivered</code> / <code>replayed</code>); <code>off</code> keeps only the floor. A compliance floor is retained at every level, even <code>off</code>: <code>viewed</code> (the HIPAA PHI-access trail) plus the terminal <code>dead</code> / <code>error</code> / <code>failed</code> . Not a master switch — it never touches the <code>messages</code> /queue disposition rows (count-and-log is separate) or the tamper-evident <code>audit_log</code> chain.
<code>audit_all_authz</code>			→ moved to <code>[security].audit_all_authorization_decisions</code> (ADR 0118) — set it there; no longer accepted in <code>[diagnostics]</code> .

Retention for the event log has its own short window — `[retention].connection_event_retention_hours` (in **hours**; `0` = inherit `messages_days`).

[egress]

Fail-closed **outbound destination allowlist** (WP-11c; ASVS 13.2.4/13.2.5/14.2.3) — bounds where the engine may **send** PHI, so a fat-fingered or hostile destination can't exfiltrate it. Each list is **opt-in**: empty = unrestricted (today's behavior); once a transport's list is set, an outbound of that transport not on it is **refused at config load/reload** (a `WiringError` → 422 / refused reload), checked against the resolved (`env()` -substituted) destination.

Key	Type	Default	Notes
<code>allowed_mllp</code>	list	<code>[]</code>	allowed MLLP destinations; each entry is <code>host</code> (any port) or <code>host:port</code> . Via <code>env</code> : comma-separated <code>MEFOR_EGRESS_ALLOWED_MLLP</code>

Key	Type	Default	Notes
<code>allowed_tcp</code>	list	<code>[]</code>	allowed raw-TCP (<code>Tcp(...)</code>) destinations; each entry is <code>host</code> (any port) or <code>host:port</code> . An inbound <code>Tcp(...)</code> is a local listener and is not gated. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_TCP</code>
<code>allowed_file_dirs</code>	list	<code>[]</code>	allowed File output directories; a destination's directory must resolve at/under one of these
<code>allowed_http</code>	list	<code>[]</code>	allowed REST/SOAP (HTTP) destination hosts; each entry is <code>host</code> (any port) or <code>host:port</code> (ADR 0003). Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_HTTP</code>
<code>allowed_db</code>	list	<code>[]</code>	allowed DATABASE destination servers; each entry is <code>host</code> (any port) or <code>host:port</code> (ADR 0003). Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_DB</code>
<code>allowed_remote</code>	list	<code>[]</code>	allowed RemoteFile (SFTP/FTP/FTPS) hosts — gates the connector in both directions (source poll + destination upload); each entry is <code>host</code> (any port) or <code>host:port</code> . Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_REMOTE</code>
<code>allowed_smtp</code>	list	<code>[]</code>	allowed email (SMTP) destination hosts for the <code>Email(...)</code> outbound (ADR 0029); each entry is <code>host</code> (any port) or <code>host:port</code> . Distinct from <code>[alerts].smtp_allowed_hosts</code> , which gates the alert notifier's own SMTP dial. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_SMTP</code>
<code>allowed_direct</code>	list	<code>[]</code>	allowed Direct (S/MIME-over-SMTP HISP relay) destination hosts (ADR 0085); each entry is <code>host</code> (any port) or <code>host:port</code> . Kept deliberately separate from <code>allowed_smtp</code> so an operator can permit a Direct HISP relay without opening generic email egress — a distinct trust relationship carrying encrypted PHI. Via env: comma-separated <code>MEFOR_EGRESS_ALLOWED_DIRECT</code>
<code>proxy_url</code>	str	<code>unset</code>	site-wide default forward/egress web proxy for the HTTP family — REST/SOAP/FHIR/ <code>fhir_lookup</code> /DICOMweb plus the OAuth2/SMART token endpoints (ADR 0126). A connection that sets no per-connection <code>proxy</code> inherits this; a per-connection value overrides it. Unset (default) = no site-wide proxy (byte-identical — only per-connection proxies apply). <code>"default"</code> selects the OS default web proxy (<code>getproxies()</code>); an <code>http(s)://</code> address names an explicit one. Proxy credentials stay per-connection (secrets via <code>env()</code>), never a global TOML value. Via env: <code>MEFOR_EGRESS_PROXY_URL</code>
<code>proxy_no_proxy</code>	list	<code>[]</code>	the site-wide <code>NO_PROXY</code> -style bypass list inherited by a connection that sets no per-connection <code>proxy_no_proxy</code> . Each entry is a host, <code>.suffix</code> , <code>*.suffix</code> or <code>*</code> . Via env: comma-separated <code>MEFOR_EGRESS_PROXY_NO_PROXY</code>
<code>deny_by_default</code>			→ moved to <code>[security].block_unlisted_outbound</code> (ADR 0118) — set it there; no longer accepted in <code>[egress]</code> .
<code>fhir_require_structured_params</code>	bool	<code>false</code>	when <code>true</code> , a <code>fhir_lookup</code> search must use the structured <code>params=</code> form (each value percent-encoded); the flat author-encoded <code>?-query</code> is refused before it dials out (ASVS 1.2.2, ADR 0043). A read-by-id and the <code>params=</code> form are unaffected. Default <code>false</code> keeps the flat form (byte-identical). Via env: <code>MEFOR_EGRESS_FHIR_REQUIRE_STRUCTURED_PARAMS</code>

FHIR-read query hardening (ASVS 1.2.2). For a Pass posture set `[egress].fhir_require_structured_params = true` so every `fhir_lookup` search is forced through the per-value-encoded `params=` form and the flat `?-query` escape hatch (which relies on the Handler author to encode each value) can no longer smuggle an extra FHIR search parameter. Default-off keeps the flat form for back-compat, which leaves the query-encoding an author responsibility — the structured form is the safe path either way.

Fully-open egress on a PHI instance is a startup REFUSAL, not a warning. With no `[egress].allowed_*` list set and deny-by-default off, `serve` **exits 2** — on **any** PHI instance, and all three built-in environment names (`dev`, `staging`, `prod`) derive PHI, not just `prod / staging` — under `[security].enforcement = enforce`, the shipped default. It downgrades to a stderr warning only under `enforcement = warn`; a **synthetic** instance is exempt. Lock it down with the per-transport lists above and/or `[security].block_unlisted_outbound = true` (the old `[egress].deny_by_default` spelling is rejected at config load).

The webhook/SMTP **alert** sinks carry no message bodies (no PHI) and keep their own host allowlists in `[alerts]` (`webhook_allowed_hosts` / `smtp_allowed_hosts`).

Cleartext (`http://`) egress is refused, whatever the data label (ASVS 12.2.1, ADR 0153). Separately from this allowlist, a plaintext `http://` outbound to a **non-loopback** host is decided by `config/tls_policy.py`'s `insecure_hop_disposition`, enforced at construction by `refuse_cleartext_egress` (`transports/rest.py`). The precedence is: a loopback / on-box hop is **allowed**; a per-connection `tls_hop_attested` hop is **allowed** (the operator asserts it is secure by other means); a per-connection `cleartext_accepted` hop is **crossed with a loud, audited WARN** (the operator accepts that it is not secure); a non-enforcing instance (`[security].enforcement = warn`) **warns**; everything else is **refused** (fail-closed). **data_class is no longer read here** — a **synthetic** label used to allow every cleartext hop silently, which made a typo in one file indistinguishable from a deliberate declaration, with every transport hop in the product as its blast radius. `MEFOR_ALLOW_INSECURE_TLS` no longer influences this decision either; it survives for the non-connection cells that have nowhere to carry a declaration.

[shadow]

Parallel-run / **shadow-instance** egress suppression (#15). A *shadow* MessageFoundry processes real (teed) traffic to validate it against a legacy engine, but must **not** deliver to live partners (the legacy engine is still the real sender). An outbound in **simulate** mode runs the full pipeline + count-and-log and finalizes the message **PROCESSED**, but **suppresses the real egress** (no bytes/SQL leave the box) and retains the would-send payload for parity comparison.

Key	Type	Default	Notes
<code>simulate_all_egress</code>	bool	false	deployment-wide master switch: when <code>true</code> , every outbound runs egress-suppressed regardless of its own <code>simulate=</code> — so a shadow stand-up can't accidentally leave one outbound live. Default <code>false</code> = each outbound's own <code>simulate=</code> flag applies.

Per-outbound control is the precise mechanism — set `simulate = true` on an individual outbound (`outbound(..., simulate=True)` or `simulate = true` in `connections.toml`); this section is the blunt instance-wide override. A simulated lane is surfaced as `simulated` on `GET /connections` and shown as `[SIMULATED]` in the console. Simulate suppresses **egress only** — the `[egress]` allowlist, connector construction, and handler state writes are unaffected. With egress suppressed there is no real partner reply, so a **capturing / reingress_to** outbound captures (and re-ingresses) **nothing** in simulate — the message just finalizes **PROCESSED**.

Simulate is not "not deployed." A simulated outbound is **fully wired** — its connector is built, its `env()` values are resolved, it receives rows, and it finalizes **PROCESSED**; it just suppresses the bytes on the wire. A **not-deployed** connection (`deployed=false`, ADR 0111) is the opposite: it is never built, its `env()` is never resolved, and a `Send` to it is recorded-and-dropped, not delivered-to-nothing. Use *simulate* for parallel-run; use *not deployed* for a feed that exists in config but is deliberately dark. See [CONNECTIONS.md](#) → [Connection lifecycle](#).

[alerts]

Where the delivery pipeline's operational alerts (e.g. `connection_stopped`, `queue_buildup`, `connection_error`, `message_stall`, `integrity_drift`) are delivered. **Both transports are off by default** — with neither configured, events are logged at **WARNING** (the `LoggingAlertSink`). A transport turns on when its essentials are present. Payloads carry the connection name + queue shape only — **never a message body** (no PHI). Delivery is best-effort and runs on a background task, so it never blocks or hangs a delivery lane.

Key	Type	Default	Notes
<code>webhook_url</code>	str	<i>unset</i>	enable the webhook transport: HTTP <code>POST</code> the event as JSON here (fronts Slack/Teams/PagerDuty/custom inbound webhooks).
<code>webhook_timeout</code>	num	10	seconds per POST
<code>webhook_allowed_hosts</code>	list	<code>[]</code>	egress allowlist for the webhook host (<code>[]</code> = any); SSRF defense (ASVS 15.3.2/1.3.6)
<code>email_smtp_host</code>	str	<i>unset</i>	SMTP server; with <code>email_from</code> + <code>email_to</code> set, enables the email transport
<code>email_smtp_port</code>	int	587	SMTP port
<code>email_from</code>	str	<i>unset</i>	sender address (required for email)
<code>email_to</code>	list	<i>unset</i>	recipient(s) (required for email). Via env: comma-separated <code>MEFOR_ALERTS_EMAIL_TO</code>
<code>email_use_tls</code>	bool	<code>true</code>	issue STARTTLS before sending
<code>email_username</code>	str	<i>unset</i>	SMTP login user (omit for unauthenticated relays)
<code>email_password</code>	str	<i>unset</i>	secret — supply via <code>MEFOR_ALERTS_EMAIL_PASSWORD</code> , never the file (or use <code>email_password_secret</code>)
<code>email_password_secret</code>	str	<i>unset</i>	connector <code>SecretProvider</code> reference (ADR 0019 §5) — when set and <code>[secrets].provider</code> is configured, the SMTP password is resolved from that backend (e.g. a Vault KV <code>path#field</code>) instead of <code>email_password</code> . A reference, not a secret.
<code>email_timeout</code>	num	30	seconds per send
<code>smtp_allowed_hosts</code>	list	<code>[]</code>	egress allowlist for the SMTP host (<code>[]</code> = any); parity with <code>webhook_allowed_hosts</code> (WP-11c)
<code>email_subject_template</code>	str	<i>unset</i>	optional operator-editable alert-email subject (#138, ADR 0127). Unset (the default, with its two siblings) = the fixed subject + key/value body, byte-identical to before. When set it is a <code>{name}</code> template over a closed non-PHI variable allow-list , validated at config load and fail-closed — an unknown or message-derived reference raises rather than rendering
<code>email_body_template</code>	str	<i>unset</i>	the same, for the plain-text body. The plain-text part is always sent, even when an HTML alternative is configured
<code>email_html_template</code>	str	<i>unset</i>	the same, adding an HTML alternative part whose substituted <i>values</i> are HTML-escaped. Never HTML-only — it supplements <code>email_body_template</code> , it does not replace it
<code>security_notifications_required</code>	bool	<code>true</code>	secure-by-default gate (BACKLOG #188, ASVS 6.3.5/6.3.7) . On a PHI instance, if no effective out-of-band security-notification channel is configured — <code>[auth].notify_security_events</code> on and <code>email_smtp_host</code> + <code>email_from</code> set — serve refuses to start (exit 2). The refuse/warn split is <code>[security].enforcement</code>, not the production tier — the gate reads <code>enforcing</code> , <code>enforce</code> is the shipped default, and all three built-in env names derive PHI, so <code>serve --env staging</code> on stock defaults with no <code>[alerts]</code> SMTP is refused, not warned. It warns only under <code>enforcement = warn</code> . Set <code>false</code> to accept the pull-only <code>GET /me/security-events</code> feed instead (audited).
<code>realert_seconds</code>	num	300	suppress re-notifying the same (event, connection) more often than this (anti-spam for a flapping lane). A matching rule's <code>cooldown_seconds</code> overrides it.
<code>rules</code>	list	<code>[]</code>	ordered <code>[[alerts.rules]]</code> table array — per-event severity, transport routing, thresholds, suppression, cooldown (see below). Empty = today's behaviour (every event → every transport at <code>warning</code>).

`[[alerts.rules]]` — per-event routing (ADR 0014)

Each rule is a row in an **ordered** `[[alerts.rules]]` array; the **first matching rule wins** (so put the most specific rules first). An event matching **no** rule keeps the default: notify **every** configured transport at `warning` with the global `realert_seconds` — so adding a rule never silently silences an event you didn't name. Matching is pure config (no code/ `eval`).

Key	Type	Default	Notes
event_type	str	any	match this event: any, connection_stopped, queue_buildup, storage_threshold, cert_expiry, connection_error, message_stall, integrity_drift, or gcm_invocations
connection	str (glob)	*	glob over the connection name (e.g. OB_*, IB_ACME_*)
min_depth	int	unset	queue_buildup only — match only when pending depth is at/over this
min_oldest_seconds	num	unset	queue_buildup only — ...or the oldest pending message has waited at least this long
severity	str	warning	info warning critical — tagged onto the event (webhook JSON + email subject) for downstream triage
transports	list	all	which transports fire: subset of ["webhook", "email"]; unset = all configured ; [] = SUPPRESS (drop silently)
cooldown_seconds	num	global	override realert_seconds for matching events (e.g. re-page a critical sooner)

```
[alerts]
webhook_url = "https://hooks.example.com/services/XXX" # webhook transport on
email_smtp_host = "smtp.example.com" # email transport on
email_from = "alerts@example.com"
email_to = ["oncall@example.com"]

# Page (webhook) immediately and re-page every minute when any inbound connection stops.
[[alerts.rules]]
event_type = "connection_stopped"
connection = "IB_*"
severity = "critical"
transports = ["webhook"]
cooldown_seconds = 60

# A deep backlog on any outbound is critical; a shallow one only emails.
[[alerts.rules]]
event_type = "queue_buildup"
min_depth = 1000
severity = "critical"

[[alerts.rules]]
event_type = "queue_buildup"
severity = "info"
transports = ["email"]

# Stay quiet about a known-bursty test feed.
[[alerts.rules]]
connection = "OB_LOADTEST"
transports = [] # suppress every event for this connection
```

A rule routing to a transport that isn't configured (e.g. `transports = ["email"]` with no SMTP settings) is rejected at startup, so a typo can't silently black-hole an alert. Severity travels in the payload; **timed multi-stage escalation** ("email now, page in 15 min") is future work (ADR 0014 §3) — rules give the static routing primitive it would build on.

[cert_monitor]

Periodic TLS-certificate **expiry monitor**. Now that native off-loopback TLS is the supported posture (`DEPLOYMENT.md`), a silently expiring certificate is a hard PHI-feed outage at renewal time. The engine scans the certs it actually serves with — the `[api].tls_cert_file` and every connection's `tls_cert_file` (MLLP server/client identity) — and raises a `cert_expiry` alert (an `[alerts]` event — route it with a `[[alerts.rules]]` rule) when one is expired or within `warn_days` of expiry. Only the **public certificate** is read (its `notAfter`), never a private key. On by default with a 30-day window; set `warn_days = 0` to disable.

Service-caller (inbound mTLS) certs — ASVS 6.4.5. A cert an inbound *caller* presents is one the engine only **verifies**, never serves, so the scan above cannot see it. Two arms cover it: the cert-identity resolver checks the `notAfter` of each verified, allow-listed client cert **at the mTLS handshake** and raises the same `cert_expiry` alert when it is inside `warn_days` (throttled per cert at the `check_interval_seconds`

cadence, since that path runs per request); and `[api].tls_client_cert_files` lets you list caller certs you hold copies of, so they are scanned like any other file. The file list is what covers a caller whose cert expires **while it has stopped connecting** — a handshake can only reveal a cert that is still being presented.

Key	Type	Default	Notes
<code>warn_days</code>	int	30	alert when a served cert expires within this many days; <code>0</code> disables the monitor
<code>check_interval_seconds</code>	num	43200	rescan cadence (default 12h); the per-cert re-alert throttle is <code>[alerts].realert_seconds</code>

```
[cert_monitor]
warn_days = 45           # start warning 45 days out

# Page (don't just email) when a served cert is close to expiry.
[[alerts.rules]]
event_type = "cert_expiry"
severity = "critical"
transports = ["webhook"]
```

[secrets] — connector SecretProvider selection

Selects **how a named connector credential is sourced** (ADR 0019 §5) — from an external secrets backend **instead of** a `MEFOR_*` env var. The connector-secret twin of `[store].key_provider` (which sources the store DEK).

Key	Type	Default	Meaning
<code>provider</code>	str	<code>none</code>	<code>none</code> env vault. <code>none</code> (default) consults no provider — every credential stays env-sourced (byte-identical). env resolves a reference as an env-var name; vault reads Vault KV v2 behind the lazy <code>[vault]</code> / <code>hvac</code> extra (the same dependency the store's Vault <code>key_provider</code> uses — no new dep). Names a <i>provider</i> , not a secret.

A provider is consulted **only** for a credential whose per-credential `*_secret` reference is set — today `[auth].ad_bind_password_secret` and `[alerts].email_password_secret` (the wired points); the SQL Server store password is seam-only (managed identity is preferred there). A reference is `"<kv-path>"` or `"<kv-path>#<field>"` for vault (field defaults to `value`; KV mount from `MEFOR_SECRETS_VAULT_KV_MOUNT`, default `secret`); Vault address/token come from `MEFOR_SECRETS_VAULT_ADDR` / `MEFOR_SECRETS_VAULT_TOKEN` (falling back to `hvac's VAULT_ADDR` / `VAULT_TOKEN`). **Fail-closed:** a reference with `provider = none`, an unknown provider, a missing `[vault]` extra, or an unresolvable/empty secret raises at load/connect — never a blank credential; the value is never logged.

[secret_rotation]

Periodic **secret-rotation reminder** (ADR 0019 §5.1) — the secret-side twin of `[cert_monitor]`. A TLS cert carries its own expiry, but a long-lived secret (the **store data-encryption key** today; connector credentials in a future `SecretProvider`) has none, so a stale key can sit unrotated with no in-engine signal. The engine periodically compares each tracked secret's operator-recorded **last-rotated date** against its **max age** and raises a **secret_rotation_due** alert (an `[alerts]` event — route it with a `[[alerts.rules]]` rule) when it is overdue or within `warn_days` of due. It reads only the rotation **dates** you configure here — **never any secret value** (PHI-free). This is a *reminder*, not enforcement: it never rotates a key or blocks startup (run `rotate-key` to rotate the store DEK). Under `[security].enforcement = enforce`, a store DEK past `store_key_max_age_days + enforce_grace_days` escalates its alert at restart (`enforced = true`) — still an alert, never a refusal.

The store DEK is tracked **live-by-default** (ASVS 13.3.4): at first keyed start the engine records a non-secret tracked-since stamp (the DEK key-id + first-seen date) in store meta and watches the DEK off it, so `store_key_last_rotated` is an **override**, not a prerequisite. The connector/AD/SMTP/Vault/OIDC credentials the engine holds are tracked too — each fingerprinted with a DEK-derived keyed MAC, its clock reset when the fingerprint changes (rotation auto-detected). A 14-day look-ahead applies once a secret is tracked; set `warn_days = 0` to disable the reminder.

Key	Type	Default	Notes
<code>warn_days</code>	int	14	alert when a tracked secret is due within this many days; <code>0</code> disables the reminder
<code>check_interval_seconds</code>	num	86400	rescan cadence (default 24h); the per-secret re-alert throttle is <code>[alerts].realert_seconds</code>

Key	Type	Default	Notes
<code>store_key_last_rotated</code>	str	—	ISO <code>YYYY-MM-DD</code> the store DEK was last rotated; unset ⇒ the DEK is still tracked live-by-default off a persisted first-seen stamp (this date is an override)
<code>store_key_max_age_days</code>	int	365	rotate the store DEK within this many days of its effective last-rotated (the operator date if set, else the persisted stamp)
<code>secret_max_age_days</code>	int	365	max age for the non-DEK tracked secret classes (connector/AD/SMTP/Vault/OIDC), alerted this many days after their last observed fingerprint change
<code>enforce_grace_days</code>	int	30	under <code>[security].enforcement=enforce</code> , a DEK older than <code>store_key_max_age_days + this</code> escalates its rotation alert at restart (still an alert, never a refusal)

```
[secret_rotation]
store_key_last_rotated = "2026-01-15" # when you last ran rotate-key
store_key_max_age_days = 365          # remind me a year later
warn_days = 30                       # start 30 days ahead

# Page when the store DEK is overdue for rotation.
[[alerts.rules]]
event_type = "secret_rotation"
severity = "warning"
transports = ["email"]
```

[cluster] — active-passive HA coordination (Track B)

Server-DB-backed. Introduces the multi-node coordination seam — a `nodes` table, a per-node heartbeat, (Track B Step 4) **leader election**, and (Step 6) **cross-node reference + config-reload convergence** — *without changing single-node behavior*. It runs as **active-passive** HA: one leader runs the whole graph, a standby takes over on failure. (The horizontal **active-active** scale-out path — per-lane ownership running the graph on every node — was **dropped (2026-06-18) and its code removed**; it is not a planned milestone.) With `enabled = false` (the default) the engine uses a no-op coordinator and runs **byte-identically** to before. Enabling it requires a **server-DB** store **and** `[store].pool_size >= 2` — a clustered node drives concurrent background work (the membership/lease-renewal maintenance loop + the per-stage workers) against the pool, so a pool of 1 would serialize everything (prefer `>= 3`). A cross-section validator refuses either violation at config load. Two backends qualify:

- postgres** — the full coordinator: leader election, the row leases, and the leader reclaim sweep, run as active-passive HA (the leader runs the graph; a standby takes over on failure).
- sqlserver** — **active-passive too**: the same self-fencing leadership lease (one leader drains the graph; a standby takes over on failure). A single active node (the leader) processes at a time, so the `reclaim_expired_leases` background sweep below applies on Postgres; on-promotion recovery covers both backends.

SQLite remains single-node (cluster coordination is refused on it).

With `[cluster].enabled` on Postgres, **leader election is built** as a **self-fencing lease** (Workstream A2): exactly one node across the cluster holds the `leader_lease` row and is the **leader** — it renews the lease every `heartbeat_seconds` (to `DB_now + leader_lease_ttl_seconds`, on the database's own clock so node clock skew is irrelevant to who may hold it), and a standby acquires only once that lease has **expired**. A leader that cannot renew within `leader_fence_timeout_seconds` (which must be `< leader_lease_ttl_seconds`) **self-fences** — it stops acting as leader *before* the lease can expire and a standby acquire it, so a network-partitioned old leader never double-processes (the split-brain guard). The leader-only **WRITE singletons** run on that one node while followers **no-op** them (reactive-by-polling, so failover is automatic on the next tick): - **[retention] purge/VACUUM/audit** — runs on the leader only. - **the lease-reclaim sweep** — the leader periodically calls `reclaim_expired_leases` (cadence `reclaim_interval_seconds`) to recover **crashed** nodes' in-flight rows (only rows whose lease has *expired*, never a live sibling's). In clustered mode the engine therefore **skips** the single-node unconditional `reset_stale_inflight` startup recovery, which would steal a live sibling's in-flight rows.

Poll-source intake is leader-gated (Track B Step 4b). A **poll** source — `file` (a watched directory), `database` (a polled table), `remote-file` (an SFTP/FTP directory) — reads a **shared external resource**: if more than one node polled it, the same file/row would be ingested twice. So only the **leader** polls a poll source. Under active-passive HA the whole graph — **listen** sources (`mllp`, `tcp`) and all the **staged-queue workers** (router / transform / delivery) alike — runs on the **leader only**; a standby binds no listeners and runs no workers (so poll-source

gating is belt-and-suspenders, and the queue's `FOR UPDATE SKIP LOCKED` + row leases serve intra-node concurrency and failover recovery rather than concurrent multi-node draining). The brief overlap during a leadership transition (the old leader's last in-flight poll vs. the new leader's first) is bounded by the same at-least-once guarantees that cover a crash mid-poll — the file-rename / row-claim atomicity and the downstream queue's idempotent handoff make a re-read a tolerated duplicate, never data loss. The worst-case transition window is bounded by the lease timing: a partitioned leader keeps polling until it self-fences (within `leader_fence_timeout_seconds`), and a standby cannot take over until the lease expires (`leader_lease_ttl_seconds`) — and `fence < TTL` guarantees the old leader has stopped first. For a `database` source the row-claim atomicity is the operator's `poll_statement` / `mark_statement` (claim with a status flag or `UPDATE ... RETURNING`); the engine owns the atomic rename only for file sources.

If the leader stops cleanly it expires its lease so a follower acquires leadership at once; if it crashes or is partitioned, the lease ages out and a follower acquires after at most `leader_lease_ttl_seconds`. **Single-node operation is unchanged** (the no-op coordinator is always leader, so every poll source always scans, runs the unconditional startup reset, and spawns no leader sweep).

Per-lane FIFO survives failover. Because the graph runs on the **leader only**, per-lane FIFO is naturally serialized by that single processor — there is no concurrent multi-node draining of a lane to reorder. Across a failover the order still has to be preserved for a lane whose head was in flight on the crashed/fenced prior leader: the ordinary FIFO claim (`claim_next_fifo`) reclaims that **stranded head** — this lane's expired-lease inflight row, back to pending **in the same transaction, before the head SELECT** — so the recovered head blocks the lane rather than being skipped, and a later row can never deliver ahead of it (the recovery does not wait on the leader's periodic sweep). This **replaced** the dropped active-active per-lane lease mechanism (the removed `lane_leases` table / per-lane ownership). The wall-clock row lease carries the NTP assumption: keep `[store].lease_ttl_seconds` comfortably above clock skew + the claim cadence. **Single-node is byte-identical** (the no-op coordinator is always leader); SQLite and SQL Server behave the same single-active-processor way.

Cross-node convergence is built (Track B Step 6). Two shared-state concerns now converge across nodes automatically: - **Reference sets** — materialize-from-source is **leader-gated** (only the leader re-reads the external file/DB source and writes the shared, versioned snapshot), and **every** node then **read-throughs** that snapshot into its own in-process read cache via the store's `converge_reference_cache` (matching on the per-set version). So the external source is read **once** per cluster and no follower is left on a stale cache — replacing the prior "every node re-syncs" model. Single-node is byte-identical: the no-op coordinator is always leader (materializes every pass) and the convergence call is a no-op on SQLite (the sole writer's cache is always current). - **Config reload** — an operator `POST /config/reload` on **one** node bumps a single-row `cluster_config` **version token**; every **other** node's config-convergence loop observes the higher version and reloads **its own** (identically-deployed) config dir to converge. The initiating node advances its applied version when it bumps, so it does **not** re-reload (no feedback loop). A `dry_run` never bumps; single-node never spawns the loop. This assumes **homogeneous config** across nodes (the token coordinates *when* to reload; each node reloads its own dir) — the same assumption as the dead-letter-missing-destinations/handlers startup sweeps.

The coordination seam is **built**: leader election (self-fencing lease), leader-gated singletons + poll-source intake, failover-safe per-lane FIFO (the stranded-head reclaim above), cross-node reference + config convergence, transform-STATE cross-node read-through (Step 6b), and the read-only `/cluster` ops API (Step 7) — a one-time startup `INFO` summarizes the operational assumptions. This is the supported **active-passive** HA model (one leader drains the graph; a standby takes over on failure), on **Postgres or SQL Server**. The horizontal **active-active** scale-out path (many nodes processing concurrently) was **dropped (2026-06-18) and its code removed** — it is not a planned milestone. On both backends, failover recovers the prior leader's in-flight rows on promotion, safe because the old leader self-fences before its lease expires.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	turn on the coordination seam; requires a server-DB store (<code>[store].backend = postgres</code> or <code>sqlserver</code>) and <code>[store].pool_size >= 2</code>
<code>node_id</code>	str	<code>unset</code>	override the auto id (<code>host:pid:hex</code>); pin for a stable identity / tests. Unset → reuses the store's lease owner-id, so node-id == owner-id
<code>heartbeat_seconds</code>	num	10	how often a node refreshes its <code>last_seen</code> heartbeat and renews its leadership lease (no separate leader-check knob). Must be > 0
<code>node_timeout_seconds</code>	num	30	a node is considered dead when its <code>last_seen</code> is older than this (the <code>/cluster/nodes</code> freshness filter). The leadership lease — not this timeout — is what transfers leadership. Must be > 0, and must exceed <code>heartbeat_seconds</code>

Key	Type	Default	Notes
<code>reclaim_interval_seconds</code>	num	30	how often the leader runs the lease-reclaim sweep that recovers crashed nodes' in-flight rows (followers no-op). Must be > 0
<code>leader_lease_ttl_seconds</code>	num	30	the leadership lease TTL (active-passive self-fencing). The leader renews to <code>DB_now + this</code> ; a standby acquires only once the lease has expired (on the DB clock, so node skew is irrelevant). Must be > 0
<code>leader_fence_timeout_seconds</code>	num	20	a leader that can't renew within this (its own monotonic clock, no DB I/O) self-fences — the split-brain guard. Must be > 0, > <code>heartbeat_seconds</code> , and < <code>leader_lease_ttl_seconds</code>
<code>acquire_delay_seconds</code>	num	0	leader-preference handicap (ADR 0096, per-node). Seconds this node waits PAST the lease-expiry time before it may take over an expired lease, so a preferred (<code>0</code>) node wins the routine take-over race. NEVER delays a renewal by the current leader, and only ever makes a node claim later — so it can't open a two-leader window. Governs take-over of an expired lease only (the first election on an empty table is a plain race). Must be <code>>= 0</code> . Surfaced per-node in <code>/cluster/nodes</code>
<code>promotable</code>	bool	true	non-promotable standby flag (ADR 0096, per-node). <code>false</code> = this node may never become leader (never inserts/takes-over/renews the lease); a node that somehow already leads steps down cleanly. Use for a warm, passive DR engine. At least one promotable node must exist or no node ever acquires the lease. <code>[dr].activate</code> cannot be combined with <code>[cluster].enabled</code> (a warm DR node is a non-promotable member, not a <code>[dr]</code> box). Surfaced per-node in <code>/cluster/nodes</code>

[backup] — scheduled DR backup / restore-verify

Engine-managed **scheduled + on-demand DR backup** of the config bundle and the SQLite store, written as one AES-256-GCM `.mfbak` archive to a local/UNC destination (#60, ADR 0049). **Opt-in:** `enabled = false` (the default) is a complete no-op. When enabled, the leader-gated `BackupRunner` (`pipeline/dr_backup.py`) takes a **consistent SQLite snapshot** (read-only against the live store — it never claims or mutates a staged-queue row), bundles the loaded `--config` dir, encrypts under the existing store DEK (ADR 0019 KeyProvider), applies keep-N retention, runs a lightweight restore-verify, and records one PHI-free `dr_backup` audit row. **No cloud target** — local / UNC only, so it adds no egress. On a **server-DB** store (Postgres/SQL Server) the `database` backup is DBA-delegated (#52): config-only, or skipped, per `config_only_on_server_db` .

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	opt-in master switch; a deployment with no <code>[backup]</code> is unaffected
<code>destination</code>	path	<code>""</code>	local or UNC destination dir (e.g. <code>D:/mefor-backups</code>). Required (non-empty) when enabled . A cloud URL (<code>s3://</code> , <code>https://</code> , ...) is rejected — there is no cloud target
<code>schedule_at</code>	str	<code>"02:00"</code>	daily local <code>"HH:MM"</code> the scheduled backup runs at (the same clock grammar as <code>[retention].vacuum_at</code>). <code>""</code> = on-demand only (the <code>messagefoundry backup CLI</code>), no scheduled pass
<code>retention_keep</code>	int	<code>7</code>	keep-N: after a successful, verified new archive, prune the oldest archives beyond the newest N at the destination. <code>0</code> = keep all. A verify- failed archive is never counted as a good backup when pruning, so a failing run can't evict the last good one
<code>snapshot_method</code>	str	<code>vacuum_into</code>	<code>vacuum_into</code> (default; takes a writer lock, sized for the off-peak schedule) or <code>online_backup</code> (low-contention, page-batched)
<code>include_config</code>	bool	<code>true</code>	bundle the loaded <code>--config</code> dir into the archive, so the cold seed is self-sufficient (store plus the config that interprets it) without assuming the DR box can reach the org's git repo
<code>verify_after_backup</code>	bool	<code>true</code>	run the lightweight restore-verify after every backup (open + <code>integrity_check</code> + row-count). On by default — a backup nobody has opened is a backup that silently doesn't restore
<code>full_restore_verify</code>	bool	<code>false</code>	the heavier verify: restore the snapshot to a throwaway temp DB and open it through the real <code>open_store</code> path. On-demand / opt-in extra, deliberately not the per-backup default
<code>config_only_on_server_db</code>	bool	<code>true</code>	on a Postgres/SQL Server store the DB backup is DBA-delegated (#52), so back up the config bundle only . <code>false</code> = skip the backup entirely on a server-DB store (not even a config-only archive)

Key	Type	Default	Notes
<code>allow_unencrypted</code>	bool	<code>false</code>	audited escape permitting a cleartext archive on a no-key synthetic instance (the parallel of <code>[security].allow_unencrypted_phi</code>). A PHI instance with no key still refuses to write an unencrypted archive regardless of this flag

[dr] — third-tier disaster-recovery standby

A **right-sized DR box** that activates only when the whole HA pair / site is gone and then runs **only the high-priority feeds** in a deliberately degraded mode — the inverse of the dropped active-active scale-out (it runs *less*, not more) (#61, [ADR 0048](#)). **Opt-in:** `enabled = false` (the default) is a complete no-op. On activation the engine cold-seeds the store from a `[backup].mfbak` archive (fail-closed if the KeyProvider/DEK is unreachable at the DR site), starts only the connections whose resolved priority tier is at or above `priority_threshold` (the rest report `status: "filtered"`), and is fenced by **acquire-VIP-or-abort**. **Activation is manual** — `POST /dr/activate`, gated by the `dr:operate` permission; no health probe ever activates it. `enabled / activate` are read at engine start. `[dr].activate` **cannot be combined with** `[cluster].enabled` (refused at load): a warm DR-site engine is a non-promotable cluster member, not a lease-contending DR box.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	is this deployment a DR standby box at all? <code>false</code> = the normal run-profile (every connection starts, subject only to ADR 0031), byte-unchanged
<code>activate</code>	bool	<code>false</code>	should this box come up under the DR run-profile on this boot — the startup activation latch, distinct from the runtime <code>POST /dr/activate</code> endpoint. Enabled but <code>activate = false</code> is <i>provisioned-but-passive</i> : it binds no priority feeds until an operator activates it. A no-op unless <code>enabled</code>
<code>activation_mode</code>	enum	<code>manual</code>	<code>manual</code> is the only built mode — the DR box promotes solely on the explicit, RBAC-gated operator action. <code>auto</code> (the box detects HA-pair loss and self-promotes) is named so a forward-looking config is explicit, but config load rejects it with a "not yet supported" error — never a silent no-op
<code>priority_threshold</code>	enum	<code>critical</code>	start only connections whose resolved priority rank is at or above this tier (<code>[delivery].priority</code> + a per-connection <code>priority=</code>). <code>critical</code> (owner-locked default) starts only the critical feeds; <code>normal</code> would also start normal-tier ones. A below-threshold connection reports <code>status: "filtered"</code> — distinct from ADR 0031 's <code>"failed"</code> . An unknown value fails config load
<code>takeover_hook</code>	str	<code>""</code>	optional operator command run before binding the priority listeners: exit 0 = "VIP acquired", any non-zero or timeout = "not acquired" and activation aborts . For an ADR 0047 load-balancer topology the passive LB is the fence and this is belt-and-braces only. <code>""</code> = no hook; a whitespace-only value is rejected at load (it would run an empty shell and "succeed")
<code>release_hook</code>	str	<code>""</code>	the symmetric command run on <code>POST /dr/release</code> to hand the VIP back to the recovered primary. <code>""</code> = no hook; whitespace-only rejected at load
<code>takeover_timeout_seconds</code>	float (>0)	<code>30.0</code>	bound on the takeover/release hook and on the KeyProvider-reachability check at the DR site: a hook or key probe that doesn't succeed within this aborts activation closed — no hang, no silent retry-forever
<code>seed_archive</code>	path	<code>""</code>	the <code>.mfbak</code> archive to cold-seed the DR store from on activation. <code>""</code> = the operator supplies the archive path in the <code>POST /dr/activate</code> request body instead (the runbook path). A cloud URL is rejected — local/UNC only, like the backup destination
<code>restore_token</code>	path	<code>""</code>	opt-in server-DB DR restore token (BACKLOG #223 , ADR 0102). A local/UNC path to a small JSON token the DBA places on the DR box recording the expected source-backup anchor of a native (Postgres/SQL Server) restore. When set, the server-DB seed gate cross-checks it against the restored database's own latest successful <code>dr_backup</code> archive — a vintage floor a bare boolean attestation cannot give: a stale or wrong native restore's latest anchor differs, so activation refuses closed . <code>""</code> (default) = off (the gate is byte-unchanged; SQLite is a no-op). A cloud URL is rejected

[approvals]

Optional **dual-control (maker-checker)** approval for high-value actions (ASVS 2.3.5) — see [SECURITY.md](#). **Off by default**; turning it on holds the listed operations for a *distinct* second approver holding `approvals:approve`, with the requester unable to approve their own.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>false</code>	turn on dual-control; off = every action executes inline as before
<code>operations</code>	list[str]	<code>["connection_purge", "dead_letter_replay"]</code>	which operations require approval; each must be a known op key (a typo is refused at startup)
<code>expiry_hours</code>	num	72	a pending request can no longer be approved after this many hours (<code>0</code> = never expires)

[integrity]

Startup **self-attestation of the installed engine wheel** (ADR 0041 D3) — a runtime in-place-tamper tripwire (`messagefoundry/integrity.py`). At startup (and on demand) the engine hashes every **loaded** first-party `messagefoundry` module file against the installed wheel's `*.dist-info/RECORD` baseline; on **drift** (a loaded module no longer matching its RECORD hash) it records a hash-chained `startup_integrity` audit row and fires the `AlertSink`. It complements ADR 0036 (which guards the `config dir`) by covering the installed *site-packages* an admin with `venv-write` + `restart` rights could edit in place. Both keys default **safe**: attestation is on but **alert-only** (it never blocks startup), and an **editable** install (`pip install -e .` — no RECORD baseline) is a **no-op**, so dev is never bricked.

Key	Type	Default	Notes
<code>enabled</code>	bool	<code>true</code>	run startup attestation at all. On by default (alert-only is harmless); a no-op off an editable install. Set <code>false</code> only to suppress the check entirely (e.g. an unusual packaging where RECORD is known-stale) — you then lose the in-place-tamper tripwire.
<code>fail_closed_on_drift</code>	bool	<code>false</code>	when <code>true</code> , drift makes <code>serve</code> refuse to start (after recording the audit row + alerting). Default <code>false</code> = alert-only : a legitimate reviewed in-place security hotfix (the documented vendored-parser patch contingency) would itself trip a RECORD mismatch, so fail-closed-by-default would brick a legitimate patch. Opt in for hard enforcement on a locked-down instance.
<code>audit_verify_on_start</code>	bool	<code>false</code>	when <code>true</code> , the engine re-walks the tamper-evident audit_log hash chain once at startup (#190). Alert-only by construction : a broken chain logs a WARNING and fires the <code>AlertSink</code> but never crashes startup — a refuse-to-start on a tripped tamper alarm would be a self-inflicted DoS. Default <code>false</code> (opt in): on a very large <code>audit_log</code> the full re-walk adds startup latency, so it is not on by default.

[engine]

Not implemented. There is **no EngineSettings model**, so an `[engine]` block in `messagefoundry.toml` is parsed and **silently dropped** (`ServiceSettings` is `extra="ignore"`). The rows below are the shape a future section would take, not knobs that do anything today. In particular `data_dir` does **not** anchor relative paths — reach for `[environments].base_dir` / `serve --project-root` for `env()` value files, and `--db` / `[store].path` for the store, instead.

Key	Type	Default	Notes
<code>shutdown_timeout_seconds</code>	int	—	accepted-but-ignored (proposed): graceful stop. The ASGI lifespan's <code>engine.stop()</code> is not bounded by a setting today
<code>data_dir</code>	str	—	accepted-but-ignored (proposed): base for relative paths. Setting it anchors nothing

[service] (NSSM / Windows)

The NSSM **install** knobs — auto-restart, stdout/stderr log paths — live in `scripts/service/`. The section here is the engine's **read-only** report of its own Windows-service run state to the console (L6a, ADR 0065): an unprivileged `sc query <service_name>` off the event loop, surfaced at `GET /service/status` and gated by `monitoring:read`. There is deliberately **no control** here — no start/stop/restart, the engine can't restart its own host over the API. Off by default, and **file-only** (`[service]` has no `MEFOR_*` env layer).

Key	Type	Default	Notes
<code>report_status</code>	bool	<code>false</code>	report the run state; off = no <code>sc query</code> ever runs and the route answers <code>state = "disabled"</code>
<code>service_name</code>	str	""	the Windows service to query. Letters, digits, space, <code>.</code> , <code>_</code> , <code>-</code> only (anything else is refused at load); empty = disabled

[security]

The **canonical, plain-language home for the high-value security posture switches** (ADR 0118). Each switch **defaults to the secure position**; loosening one is deliberate and **warned at serve** (see SECURITY-LOOSENING.md for what each opt-out gives up + its ASVS/NIST/HIPAA mapping). This section **replaces** the scattered legacy keys — setting a moved key in its old section (`[api].host` , `[api].serve_ui` , `[api].public_origin` , `[auth].enabled` , `[auth].require_mfa` , `[auth].session_idle_timeout_minutes` , `[auth].session_absolute_hours` , `[store].allow_unencrypted_phi` , `[egress].deny_by_default` , `[retention].messages_days` , `[retention].allow_unbounded_phi` , `[diagnostics].audit_all_authz` , `[ai].data_class` , `[ai].production`) is **rejected at load** with a pointer to its `[security]` replacement. Low-level *plumbing* (TLS cert paths, `[egress].allowed_*` contents, `[retention].dead_letter_days` , DB identity, password policy, rate limits, AD/LDAP) stays in its functional section.

Under the hood the loader **desugars** `[security]` into those internal fields, so every serve gate + the `checks.py` commit/CI mirror keep enforcing exactly as before — **no shipped refusal is loosened** (the *No-loosen rule*, ADR 0092 §5), and a PHI weakening under **strict enforcement** (`enforcement = enforce` , the default) still fails closed regardless of any value here — byte-identical to the former production-PHI refusal (ADR 0148).

key	type	default	meaning
<code>local_access_only</code>	bool	<code>true</code>	reachable only from this machine (loopback bind)
<code>listen_address</code>	str	<code>"127.0.0.1"</code>	bind address — used only when <code>local_access_only = false</code>
<code>require_encryption_for_remote</code>	bool	<code>true</code>	any off-machine access must be over TLS (config-file twin of <code>--allow-insecure-bind</code> ; can't relax production-PHI)
<code>serve_web_console</code>	bool	<code>true</code>	mount the browser ops console at <code>/ui</code> — on by default (ADR 0143); set <code>false</code> to shrink to a JSON-only surface. Default-on applies to local loopback binds; on an exposed instance a default-on console auto-degrades to JSON-only unless explicitly enabled with TLS + <code>web_console_public_address</code>
<code>web_console_public_address</code>	str	<code>""</code>	external origin when the console is exposed off-box (CSRF/CSWSH + WebAuthn RP-id)

key	type	default	meaning
allowed_client_networks	list[str]	[]	[BUILT] (ADR 0151): source-address allow-list for the operator API + web console. Empty (the default) = no restriction. Non-empty = a request whose client address is outside every listed network is refused 403 in middleware, before routing and before sign-in (also covers <code>/ui</code>, <code>/ui/static</code>, <code>/ws/stats</code>). Entries are CIDR networks or bare hosts (<code>"10.20.0.0/16"</code>, <code>"2001:db8::/48"</code>, <code>"10.20.4.7" → /32</code>), IPv4 + IPv6 mixed; malformed entries are refused at load and valid ones are stored normalized (<code>10.1.2.3/24 → 10.1.2.0/24</code>). Loopback is always allowed, with no knob (the tray <code>/health</code> poll, an on-box browser, <code>messagefoundry check</code> and a container HEALTHCHECK cannot be allow-listed). Operator surface only — the ingest listeners keep their own per-connection <code>source_ip_allowlist</code> (an attribute on the connection in <code>connections.toml</code>, not a key in the <code>[inbound]</code> section — put it there and it is silently ignored). It matches the address uvicorn reports, so it is INERT behind an UNDECLARED proxy / NAT / a bridged container — declare the proxy in <code>[api].trusted_proxies</code> or this does nothing; <code>curl /health</code> and read <code>observed_client</code> to check. Setting it tightens <code>[api].trusted_proxies</code> to single hosts (a broad range would let every host inside it forge its own source address). Startup-only: a lockout costs a service restart. Defence-in-depth behind the host firewall, not the primary network control — read OFF-LOOPBACK-DEPLOYMENT.md first. Env: <code>MEFOR_SECURITY_ALLOWED_CLIENT_NETWORKS</code> (comma-separated).
encrypt_stored_data	bool	true	PHI encrypted at rest (key from the environment)
allow_unencrypted_phi	bool	false	audited escape: start a PHI instance with no key
allow_unencrypted_phi_under_strict_enforcement	bool	false	the second acknowledgment required to start a PHI instance keyless under strict enforcement (ADR 0140). Under <code>enforcement = enforce</code>, <code>allow_unencrypted_phi = true</code> on its own is not enough — <code>serve</code> still refuses to start (exit 2) unless this is also set, so the highest-risk posture (real PHI + strict enforcement) is never one flag away from plaintext at rest. Under <code>enforcement = warn</code> the single <code>allow_unencrypted_phi</code> flag still governs. With both set the instance starts with PHI bodies, summary/metadata and the error columns unencrypted at rest, and the startup AUDIT line names both flags. A loosening — <code>security_loosenings()</code> reports it, so it is never silent

key	type	default	meaning
allow_single_factor_admin_when_exposed	bool	false	permit single-factor admin on an exposed PHI instance (ADR 0140). With <code>require_sign_in</code> on, <code>require_mfa</code> explicitly off, and the operator surface exposed (a non-loopback bind, or the console reached through a declared TLS-terminating proxy), a PHI instance under <code>enforcement = enforce</code> refuses to start (exit 2) — the Administrator role would authenticate with a single factor over the network. Setting this permits that start; it is recorded in a WARNING-level AUDIT line and the ordinary exposure warning still prints. A loosening — <code>security_loosenings()</code> reports it. Prefer <code>require_mfa = true</code>: it gates only local Administrator accounts (directory MFA is delegated), so it is safe to leave on even on an AD-only deployment
memory_encryption_operator_declared	bool	false	[BUILT] (ADR 0152 rung 2, ASVS 11.7.1): the operator's declaration that this host provides hardware memory encryption (AMD SEV-SNP / Intel TDX), so PHI is protected in RAM while it is being processed. The engine cannot verify it — a local CPU flag is emitted by the OS whose integrity the requirement protects against — so this records who took responsibility, the same discipline as <code>MEFOR_TLS_REVOCATION_ATTESTED</code>. It is deliberately not called "attested": in confidential computing that word means a CPU-signed quote verified against the silicon vendor's root PKI (ADR 0152 rung 3, not built). An exposed PHI instance without it warns and starts — on every environment, at both <code>enforcement</code> settings; it refuses only if <code>require_memory_encryption_declaration</code> is also set. A positive platform read-out does not substitute for it (a read-out must never relax a control). Loopback and synthetic instances are byte-identical (never consulted). If the platform read-out positively contradicts this, the contradiction is warned at start and reported as <code>memory_encryption_readout_contradicts_declaration</code> on <code>GET /security/posture</code> — but never refused (the read-out is a self-report, not evidence, and has known false negatives: driver not loaded, container without the device node mapped, Azure CVM paravisor). Setting this does not make the instance ASVS 11.7.1-compliant — see the read-out note below the table. Env: <code>MEFOR_SECURITY_MEMORY_ENCRYPTION_OPERATOR_DECLARED</code>

key	type	default	meaning
<code>require_memory_encryption_declaration</code>	bool	false	[BUILT] (ADR 0152 rung 2): turn the row-12 warning above into a refusal — an exposed PHI instance with no <code>memory_encryption_operator_declared</code> then refuses to start under <code>enforcement=enforce</code> (and still warns under <code>warn</code>). Opt-in by design, and the default is load-bearing: the property is a host property that no operator can satisfy on Windows (the read-out is always <code>null</code> there), and "exposed" includes the recommended loopback-behind-proxy topology, so a refusal by default would stop working dev/staging/prod deployments from booting on upgrade over something they cannot change. Same scoping rule as <code>[security].allowed_client_networks</code> ' companion refusal (ADR 0151): a new refusal fires only on a new opt-in. Set it in an estate that has standardized on confidential-computing hosts and wants a missing declaration to be fatal. Env: <code>MEFOR_SECURITY_REQUIRE_MEMORY_ENCRYPTION_DECLARATION</code>
<code>require_sign_in</code>	bool	true	authenticate every request
<code>require_mfa</code>	bool	true	second factor (native TOTP or a WebAuthn passkey), enforced as an access gate since ASVS 6.3.3 — an MFA-pending session is refused on <i>every</i> authorized route with <code>403 + X-MFA-Required: 1</code> , and a browser session is confined to <code>/ui/mfa</code> .
<code>require_mfa_scope</code>	"administrators" "every_local_account"	"every_local_account"	Which local accounts must ENROL a factor when <code>require_mfa</code> is on (ASVS 6.3.3). An account that has already enrolled one must always satisfy it, under either value — this dial only decides who is required to enrol in the first place. <code>administrators</code> restores the pre-6.3.3 posture and is reported as a loosening on <code>GET /security/posture</code> (advisory, not a refusal: refusing to boot on it would break every existing deployment on upgrade). Directory (AD/Kerberos) identities are out of scope under either value — their MFA is delegated to the directory. Operator note: under the default a non-interactive local bearer-token service account becomes MFA-pending and cannot enrol unattended — move it to mTLS (<code>require_service_cert</code> , exempt by design) or to AD, or set this to <code>administrators</code> . Env: <code>MEFOR_SECURITY_REQUIRE_MFA_SCOPE</code>
<code>sign_out_after_idle_minutes</code>	int	30	session idle timeout
<code>max_session_hours</code>	int	12	session absolute lifetime
<code>block_unlisted_outbound</code>	bool	true	deny-by-default egress — only allow-listed destinations send
<code>delete_message_bodies_after_days</code>	int	30	bounded PHI-body retention; <code>0</code> = keep indefinitely (audited)
<code>allow_keeping_phi_indefinitely</code>	bool	false	audited escape: unbounded PHI retention
<code>audit_all_authorization_decisions</code>	bool	false	ePHI access is always audited regardless; this adds full authz tracing (off by default — forcing it on risks flooding the audit log)

key	type	default	meaning
<code>handles_real_patient_data</code>	bool	<i>derived</i>	the master data-class lever (was <code>[ai].data_class = "phi"</code>). Unset $\Rightarrow$ derived from the environment name — all three built-in names (<code>dev</code> / <code>staging</code> / <code>prod</code>) now derive PHI (ADR 0148 GIVEN 1, so the default/CI path exercises the encryption/egress/retention controls rather than first meeting them in production); a genuinely-synthetic dev/CI box must set <code>false</code> explicitly (a loud, audited opt-out), and a custom-named env must declare it
<code>enforcement</code>	<code>enforce</code> <code>warn</code>	<code>enforce</code>	the serve-gate refuse/warn dial + the ADR 0092 escape-clamp key (ADR 0148 GIVEN 2). <code>enforce</code> (default) refuses every PHI serve-gate violation and shuts every blunt escape-clamp — byte-identical to the former production-tier behaviour; <code>warn</code> logs + audits + continues and honours the escapes (a loud, audited loosening, named by <code>security_loosenings()</code>). Decoupled from <code>production_instance</code> (env <code>MEFOR_SECURITY_ENFORCEMENT</code>)
<code>production_instance</code>	bool	<i>derived</i>	production-tier posture (was <code>[ai].production</code>). Derived from the environment name when unset (<code>prod</code> $\rightarrow$ yes; <code>dev</code> / <code>staging</code> $\rightarrow$ no). Informational since ADR 0148 — drives the AI data-scope ceiling, the DEBUG-log refusal, and reporting, not the serve-gate refuse/warn dial (that is <code>enforcement</code>)

Editing is IDE-only: the VS Code extension's *Edit Security Settings* command (which shells `messagefoundry security show|set`) is the sole authoring surface. The **web console is read-only** — the effective posture, active loosening, and the synthetic-relaxation notice are surfaced at `GET /security/posture` (authenticated, `monitoring:read`). Authentication & RBAC *plumbing* remains in `[auth]` (see SECURITY.md); the at-rest-encryption *key* is a secret supplied via `MEFOR_STORE_ENCRYPTION_KEY` / `[store].encryption_key_file` (PHI.md).

The section is read at engine **startup**, so an edited switch takes effect on the next engine restart (`POST /config/reload` re-runs the `--config` graph, not `[security]`).

The memory-encryption read-out is *not* a compliance claim

`GET /security/posture` also carries a **report-only** platform read-out beside the FIPS attestation (ADR 0152 rung 1): `memory_encryption_self_reported_capability`, `memory_encryption_self_reported_active`, `memory_encryption_self_reported_mechanism` and `memory_encryption_readout_source`. On Linux these come from `/proc/cpuinfo` flags (**capability** — "this silicon *can*") and guest device-node presence `/dev/sev-guest` / `/dev/tdx-guest` (**activation** — "this guest *is*"), which are deliberately reported as **separate fields** and never derived from one another. On **Windows every field is `null`** (undeterminable): the in-guest attestation path is an ADR 0152 spike that has not landed, and guessing would be worse than saying so.

No value of any of these fields satisfies ASVS 11.7.1, and none may be cited as though it did. They are values the **host OS emits about itself**, and 11.7.1 exists precisely because that host may be the adversary — a compromised kernel or hypervisor forges every one of them. Only a **CPU-signed attestation report verified against the silicon vendor's root PKI** would be evidence; that is ADR 0152 rung 3 and is **not built**. Treat the read-out as configuration confirmation, never as proof.

The response says so itself. Every posture body carries `memory_encryption_note` — the same sentence the startup warning prints — so the disclaimer travels with any copy of the artifact rather than living only here. Two more fields are deliberately shaped so they cannot be quoted as compliance: `memory_encryption_operator_declared` (named for what it is: an operator's word, not an attestation) and `memory_encryption_readout_contradicts_declaration`, which is **tri-state**. `null` on that field means *nothing was measured that could contradict anything* — the answer on Windows, on an AMD SME / Intel TME host (memory-controller-wide encryption, which has no guest interface to find), in a container without the device node mapped, and whenever nobody declared anything. A `false` means the read-out **agrees**, and it is never emitted by vacuity.

The property itself is a host requirement, not a switch. `memory_encryption_operator_declared = true` records a claim; it does not create memory encryption. An ASVS **Level 3** PHI deployment must actually run the engine as a **confidential guest** on an AMD SEV-SNP or Intel TDX host — [SYSTEM-REQUIREMENTS.md](#) states the requirement and the (verified) availability picture, which today is **not reachable for a Windows guest on on-premises Hyper-V or ESXi**. On a host that does not provide the property, the honest configuration is **not** to set this: leave it unset, keep the startup warning, and disclose 11.7.1 as **Partial**. Reaching for `[security].enforcement = warn` is the wrong lever — that is the global refuse/warn dial and downgrades every other posture refusal at the same time; nothing about this control requires it, because it never refuses unless you opt in via `require_memory_encryption_declaration`. The step-by-step is in OFF-LOOPBACK-DEPLOYMENT.md § *In-use data protection*.

Example

```
# messagefoundry.toml
[store]
backend = "sqlserver"
server = "sql01.hospital.local"
database = "MessageFoundry"
auth = "sql"
username = "mefor_service"
encrypt = true

[security]
local_access_only = true           # loopback bind (ADR 0118; the bind host lives here, not [api])
delete_message_bodies_after_days = 30 # the PHI-body window (ADR 0118; NOT [retention].messages_days)

[api]
port = 8765

[logging]
level = "info"
format = "json"           # structured stdout (one JSON object per line)
# Setting forward_host turns forwarding ON by default (ADR 0080); forward_enabled = false opts out.
forward_host = "siem.hospital.local" # ship a copy off-box to a syslog/SIEM collector
forward_port = 6514                 # RFC 5425 syslog-over-TLS default
forward_protocol = "tls"           # udp (default) | tcp | tls (native RFC 5425, no agent)
forward_tls_ca_file = "C:/mefor/siem-ca.pem" # required for tls unless forward_tls_verify = false
# Opt-in startup clock-sync gate (ASVS 16.2.2) – warns on skew; add fail-closed to refuse start:
# require_time_sync = true
# ntp_peer = "ntp.hospital.local"

[retention]
# The inbound-body window is [security].delete_message_bodies_after_days above – setting
# messages_days here is REJECTED at load (ADR 0118). Only the plumbing keys stay in this section:
dead_letter_days = 90 # null dead-letter bodies after 90 days
vacuum_at = "03:30"   # daily off-peak VACUUM to reclaim space (SQLite only)

# secret via env (never in the file)
set MEFOR_STORE_PASSWORD=...
```

Build order (incremental)

1. **Done** — `ServiceSettings` model + loader (file + env + CLI precedence); `[api]` / `[logging]` and `[store]` `backend=sqlite|path|synchronous` wired into `serve` (`--service-config` + the `--db` / `--host` / `--port` / `--log-level` overrides).
2. **Done** — `[delivery]` defaults feed the default `RetryPolicy` (`DeliverySettings.retry_policy()`), alongside the ordering / internal-error / buildup-stall-saturation / priority defaults an outbound inherits when it declares none.
3. **Done** — `[store]` server-DB keys landed with the SQL Server **and** Postgres backends (both consume `server` / `database` / `username` / `pool_size` / `command_timeout` / `ssl_root_cert`); they are **implemented**, not accepted-but-ignored.
4. **Done** — `[retention]` purge/maintenance job (body-null + WAL/VACUUM, audited; `audit_days` reserved). `[logging]` structured-JSON `format` + off-box `forward_*` syslog shipping land (sec-offbox-log); PHI redaction is an always-on handler filter (no structlog).

Open decisions (to confirm)

-  **Decided and built** — **TOML file + env + CLI** as above, chosen for consistency with `pyproject.toml` and ops-friendliness; secrets via `env`.
-  **Settled** — **the IDE authors, the web console reads**. Settings are **edited from the VS Code extension** (its *Edit Security Settings* command shells `messagefoundry security show|set`, which owns the TOML write) or by hand in `messagefoundry.toml`; there is **no settings-write API**. The web console is **read-only** on posture — `GET /security/posture` (authenticated, `monitoring:read`) surfaces the effective switches and active loosening. This is the reverse of the split originally sketched here.
- Whether per-connection overrides (e.g. a connection's own retry) stay in code (today) or also move into settings. Recommendation: **keep per-connection logic in code**, service settings are defaults.

© 2026 **MEFOR-ORG**, a non-profit · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated July 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MEFOR-ORG. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.